

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра радіоелектронних і комп'ютерних систем

Допустити до захисту
Завідувач кафедри
_____ проф. Оленич І.Б.
«___» _____ 2024 р.

Кваліфікаційна робота

Бакалавр

(освітній ступінь)

**РОЗРОБКА СИСТЕМИ ПЕРСОНАЛІЗОВАНИХ РЕКОМЕНДАЦІЙ ДЛЯ
ВИБОРУ ВІДЕОПРОДУКЦІЇ З ЗАСТОСУВАННЯМ МЕТОДІВ
МАШИННОГО НАВЧАННЯ**

Виконав:
студент IV курсу групи ФЕІ-44
спеціальності 122 – Комп'ютерні науки
_____ Гальчишак Я. М.
Науковий керівник:
_____ асист. Рабош Р. В.
«___» _____ 2024 р.
Рецензент:
_____ доц. Середницька Х. І.

Львів 2024

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені ІВАНА ФРАНКА

Факультет _____ електроніки та комп'ютерних технологій
Кафедра _____ радіоелектронних і комп'ютерних систем
Освітній ступінь _____ бакалавр
Галузь знань _____ 12 "Інформаційні технології"
(шифр і назва)
Спеціальність _____ 122 "Комп'ютерні науки"
(шифр і назва)

«ЗАТВЕРДЖУЮ»

Завідувач кафедри РКС
проф. Оленич І.Б.
«__» _____ 2024 р.

З А В Д А Н Н Я

НА КВАЛІФІКАЦІЙНУ (БАКАЛАВРСЬКУ) РОБОТУ СТУДЕНТУ

Гальчишаку Ярославу Михайловичу

(прізвище, ім'я, по батькові)

1. Тема роботи "Розробка системи персоналізованих рекомендацій для вибору відеопродукції з застосуванням методів машинного навчання"

керівник роботи Рабош Роман Васильович, _____
(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджена Вченою радою факультету від "30" жовтня 2023 р. протокол №42/23

2. Строк подання студентом роботи _____ 1 червня 2024р.

3. Вихідні дані до роботи

3.1. Моделі і методи прогнозування рекомендацій для колаборативних рекомендаційних систем: <https://science.lpnu.ua/sites/default/files/journal-paper/2019/feb/15581/181912maket-68-75.pdf>

3.2. Django documentation: <https://docs.djangoproject.com/en/5.0/>

4. Зміст розрахунково-пояснювальної записки (перелік питань, які потрібно розробити)

4.1. Огляд літератури за темою дипломної роботи та освоєння необхідних для виконання проєкту комп'ютерних технологій

4.2. Аналіз алгоритму машинного навчання, для розробки системи персоналізованих рекомендацій

4.3. Написання програмного коду та його тестування

4.4. Аналіз результатів та оформлення дипломної роботи

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)
Презентація в PowerPoint

6. Консультанти розділів роботи

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання 15.11.2023р

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів кваліфікаційної (бакалаврської) роботи	Строк виконання етапів роботи	Примітка
1.	Огляд літератури за темою дипломної роботи та освоєння необхідних технологій	до 28.01.2024р.	
2.	Аналіз алгоритмів машинного навчання, для розробки системи персоналізованих рекомендацій	до 05.02.2024р.	
3.	Написання програмного коду та його тестування	до 01.05.2024р.	
4.	Аналіз результатів та оформлення дипломної роботи	до 01.06.2024р.	

Студент

_____ (підпис)

Гальчишак Я. М.

(прізвище та ініціали)

Керівник роботи

_____ (підпис)

Рабош Р. В.

(прізвище та ініціали)

АНОТАЦІЯ

Мета дипломної роботи полягає у аналізі та розробці системи персоналізованих рекомендацій, на основі вибору відеопродукції із застосуванням методу машинного навчання. Сучасні технології значно змінили спосіб споживання контенту, і ефективні рекомендаційні системи відіграють ключову роль у наданні користувачам індивідуалізованих порад щодо вибору відеопродукції. У роботі розглядається алгоритм колаборативної фільтрації, який базується на користувацькому підході, тобто на аналізі взаємодії користувачів з контентом та врахуванні їхніх уподобань для формування рекомендацій.

Велика частина дослідження присвячена аналізу існуючих методів машинного навчання, та вибору найвідповіднішого методу у практичному застосуванні для персональних рекомендацій.

Під час роботи було проаналізовано усі потреби користувачів аналогічних сервісів, і виділено основні моменти:

- Дружній та зручний інтерфейс
- Доступ до більшості фільмів
- Можливість зберігати переглянуту відеопродукцію та виставляти оцінки
- Отримання персоналізованих рекомендацій на основі вже переглянутої відеопродукції

Для зручності виконання поставленого завдання було використано TMDb API, це безкоштовний API, який дає змогу отримати докладну інформацію про будь який фільм, у вигляді JSON-відповіді.

Для створення рекомендаційної системи були використані реальні дані, отримані з популярного відео-сервісу.

ABSTRACT

The aim of the thesis is to analyze and develop a system of personalized recommendations based on the selection of video productions using the machine learning method. Modern technologies have significantly changed the way content is consumed, and efficient recommender systems play a key role in providing users with personalized advice on choosing video products. The paper considers a collaborative filtering algorithm based on a user approach, that is, on the analysis of user interaction with content and taking into account their preferences for the formation of recommendations.

A large part of the research is devoted to the analysis of existing methods of machine learning, and the choice of the most appropriate method in practical application for personal recommendations.

During the work, all the needs of users of similar services were analyzed, and the main points were highlighted:

- Friendly and convenient interface
- Access to most movies
- Ability to save viewed video productions and rate them
- Receive personalized recommendations based on already viewed video productions

To facilitate the task, TMDB API was used, it is a free API that allows you to get detailed information about any movie in the form of a JSON response.

Real data obtained from a popular video service was used to create a recommendation system.

ЗМІСТ

ПЕРЕЛІК СКОРОЧЕНЬ	7
ВСТУП.....	8
РОЗДІЛ 1. АНАЛІЗ СТАНУ ПРОБЛЕМНОЇ ОБЛАСТІ	10
1.1 Сучасний стан сфери розваг	10
1.2 Аналіз існуючих методів машинного навчання	10
1.2.1 Неконтрольоване навчання	12
1.2.2 Напівконтрольоване навчання	13
1.2.3 Навчання з підкріпленням	13
1.2.4 Контрольоване навчання	14
1.2.4.1 Колаборативна фільтрація.....	15
1.2.4.2 Фільтрація вмісту	19
1.2.4.3 Фільтрація контексту	21
РОЗДІЛ 2. АНАЛІЗ ВИКОРИСТАНИХ ТЕХНОЛОГІЙ.....	23
2.1 Python-фреймворк Django	23
2.2 Шаблонізатор	25
2.3 TMDb API	27
2.4 Система управління базами даних	28
2.5 Git та GitHub.....	29
РОЗДІЛ 3. ОПИС РЕАЛІЗАЦІЇ ПРОЄКТУ	31
3.1 Серверна частина	31
3.2 Рекомендаційна система	36
3.3 Користувачька частина	40
3.4 Майбутній розвиток проєкту	47
ВИСНОВКИ.....	49
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	50
ДОДАТОК А	52

ПЕРЕЛІК СКОРОЧЕНЬ

БД	База даних
СУБД	Система управління базами даних
AI	Artificial Intelligence (укр. Штучний інтелект)
API	Application Programming Interface (укр. Прикладний програмний інтерфейс) – набір визначень підпрограм та протоколів взаємодії для створення програмного забезпечення
IDE	Integrated Development Environment (укр. Інтегроване середовище розробки) – комплекс програмних засобів, використовуваний програмістами розробки програмного забезпечення.
ML	Machine learning (укр. Машинне навчання)
ORM	Object-relational mapping (укр. Об'єктно-реляційна проекція) – технологія, що зв'язує бази даних з концепціями ООП
TMDB	The Movie Database (укр. База даних фільмів), безкоштовна база даних фільмів і серіалів, що надає доступ до свого API
URL	Uniform Resource Locator (укр. Уніфікований локатор ресурсів) – стандартизована адреса ресурсу в інтернеті.

ВСТУП

Зі збільшенням кількості контенту, доступного для перегляду в інтернеті, користувачі часто стикаються з проблемою вибору. Щодня додаються нові фільми та серіали, що робить процес вибору складнішим. Традиційні методи пошуку та вибору відеопродукції стають все менш ефективними. Персоналізовані рекомендації можуть значно спростити цей процес, зменшуючи час, витрачений на пошук, та збільшуючи задоволення від перегляду.

Метою цього проєкту є розробка рекомендаційної системи для вибору відеопродукції, що базується на уподобаннях та переглядах користувача. Це включає створення як базової гостьової сесії, яка дозволяє новим користувачам отримати швидкі рекомендації, так і розширених функцій для авторизованих користувачів.

Оскільки середньостатистичний користувач не бажає одразу реєструватись на невідомий для нього сайт, буде розроблена гостьова сесія, яка буде відображати собою, маленьку частинку даного проєкту. Щоб зацікавити користувача до подальшої реєстрації, у гостьовій сесії буде можливість отримати «швидку рекомендацію», де користувачу буде запропоновано список із 20 популярних на даний момент фільмів.

Авторизований користувач буде мати більший перелік можливостей:

- Додавання фільмів до списку переглянутих
- Виставлення оцінок
- Отримання персональних рекомендацій на основі переглянутих фільмів
- та інші деталі

Щодня ми проводимо значну частину свого часу в інтернеті, та залишаємо за собою велику кількість інформації, яку влучно використовують алгоритми машинного навчання, спрямовані на створення надокучливих рекламних банерів, які наче знають про що ми думаємо. Однак, все набагато простіше, за цим стоїть ретельний аналіз онлайн-активності, що дозволяє передбачати наші уподобання. У цьому контексті, об'єктом дослідження в даній роботі будуть методи машинного навчання, які можуть застосовуватись для важливої та корисної мети — створення персоналізованих рекомендацій відеопродукції.

Предметом дослідження стане застосування колаборативної фільтрації для створення персоналізованих рекомендацій фільмів. Система буде інтегрована у веб-додаток, що дозволить користувачам отримувати рекомендації відповідно до їх смаків.

Для реалізації поставлених цілей у роботі буде використано: методи розробки веб-додатків та інтеграції API, методи аналізу літературних джерел, мови програмування Python та JavaScript, високорівневий Python-фреймворк Django, IDE Visual Studio Code, веб-сервери та бази даних.

Новизною у реалізації даного проєкту є TMDb API, саме з допомогою його запитів будуть отримуватись дані про фільми та виводитись списки фільмів для формування швидких і персональних рекомендацій користувача. Дане API, дозволяє робити будь які запити, від списку популярних фільмів, до деталей фільму лише по його ID.

Після реалізації проєкту, можливе розширення функціоналу системи, а саме створення: власних публічних списків з відеопродукцією, сторінок з акторами та їх фільмами, форуму для обговорення між користувачами, також можлива розробка мобільного додатку та покращення або оновлення алгоритму машинного навчання.

Підсумовуючи, цей проєкт пропонує покращення для сфери розваг, а саме систему персоналізованих рекомендацій для вибору відеопродукції. Розробка ґрунтується на методах машинного навчання, що робить її потужним інструментом для покращення користувацького досвіду та задоволення потреб глядачів.

РОЗДІЛ 1. АНАЛІЗ СТАНУ ПРОБЛЕМНОЇ ОБЛАСТІ

1.1 Сучасний стан сфери розваг

Зростання популярності онлайн-сервісів з перегляду відео, таких як Netflix, YouTube, Amazon Prime Video, Disney+ тощо, призвело до значного збільшення доступного контенту. Це в свою чергу, створює проблему для користувачів, яким стає все складніше знаходити відео, що відповідає їхнім інтересам. Це може бути особливо важливо для тих, хто має обмежений час або шукає конкретний тип контенту.

Кожен користувач має свої унікальні вподобання, інтереси та потреби. Однак існуючі платформи зазвичай пропонують загальні рекомендації, які не завжди відповідають індивідуальним потребам користувачів. Важливо, щоб рекомендації були не лише персоналізованими, а й відповідали високим стандартам якості та були релевантними для користувача. Недостатня якість рекомендацій може призвести до розчарування користувачів і зниження їхньої вірогідності використовувати платформу.

Таким чином, зростання кількості доступного контенту вимагає від онлайн-сервісів постійного вдосконалення своїх рекомендаційних алгоритмів, щоб забезпечити користувачам якісний та релевантний перегляд. Це не тільки підвищить задоволеність користувачів, але й сприятиме довгостроковій лояльності до платформи [1].

1.2 Аналіз існуючих методів машинного навчання

Машинне навчання (ML - Machine learning) — це розділ штучного інтелекту (AI - Artificial Intelligence) та інформатики, який зосереджується на використанні даних і алгоритмів для імітації процесу навчання людей. Головна мета машинного навчання — поступово підвищувати точність і продуктивність моделей, що навчаються на основі даних.

Процес машинного навчання складається з кількох основних етапів, кожен з яких відіграє важливу роль у створенні ефективної моделі. Університет Каліфорнії в Берклі (UC Berkeley) пропонує структуру, яка допомагає зрозуміти, як алгоритми машинного навчання вчать і приймають рішення. Ця структура включає три

основні частини: процес прийняття рішення, функція помилок і процес оптимізації моделі.

На першому етапі алгоритм машинного навчання приймає рішення на основі вхідних даних. Це можуть бути як дані з мітками, так і без міток. Алгоритм аналізує дані і намагається виявити шаблони або закономірності, що дозволяють зробити прогнози або класифікації.

Другий етап включає оцінку точності моделі за допомогою функції помилок. Ця функція дозволяє алгоритму визначити, наскільки його прогнози відрізняються від фактичних результатів. Якщо алгоритм має доступ до відомих прикладів, функція помилок порівнює його прогнози з цими значеннями і обчислює різницю між ними.

Третій і завершальний етап — це процес оптимізації моделі. На цьому етапі алгоритм намагається мінімізувати помилки, вносячи коригування у вагові коефіцієнти. Мета цього процесу — зробити модель якомога точнішою, щоб вона могла краще відповідати даним у навчальному наборі. Алгоритм повторює цей ітеративний процес "оцінки та оптимізації", постійно оновлюючи ваги і покращуючи свої прогнози.

Таким чином, машинне навчання — це складний, багатокроковий процес, який дозволяє алгоритмам поступово вдосконалювати свої прогнози і стає дедалі точнішим [2].

В зальному машинне навчання можна поділити на чотири основні типи: контрольоване навчання (supervised learning), неконтрольоване навчання (unsupervised learning), напівконтрольоване навчання (semi-supervised learning) та навчання з підкріпленням (reinforcement learning).

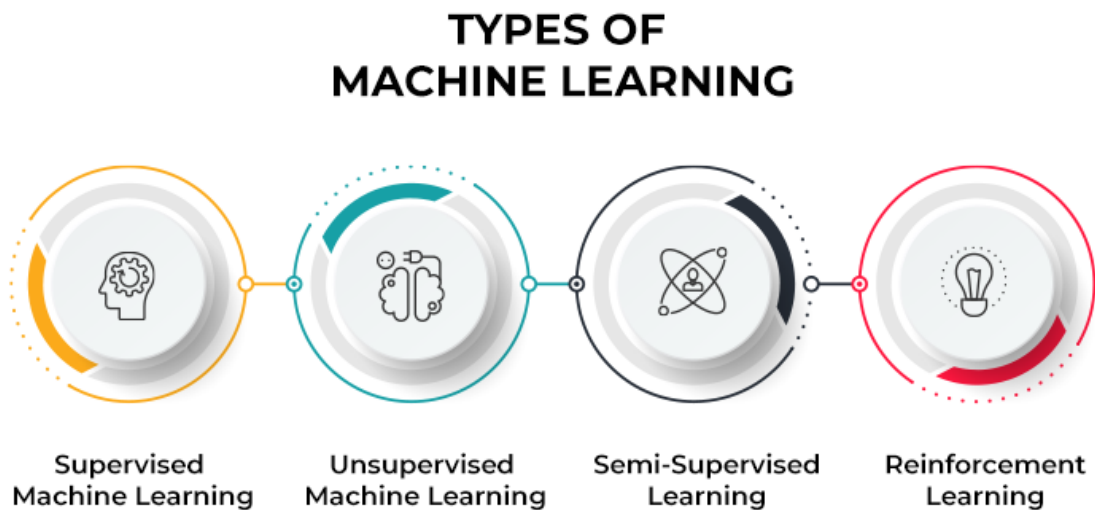


Рисунок 1.1 Основні види машинного навчання

1.2.1 Неконтрольоване навчання

Навчання без нагляду – тип машинного навчання, який вивчає дані без нагляду людини. Моделям машинного навчання надаються дані без міток і їм дозволяється виявляти закономірності та ідеї без будь-яких явних вказівок чи інструкцій. Здатність цього методу виявляти подібності та відмінності в інформації робить його ідеальним для дослідницького аналізу даних, стратегій перехресних продажів, сегментації клієнтів, а також для розпізнавання зображень і шаблонів. Аналіз головних компонентів і сингулярне розкладання є двома поширеними підходами для цього. Інші алгоритми, що використовуються в неконтрольованому навчанні, включають нейронні мережі, кластеризацію k-середніх і ймовірнісні методи кластеризації [3].

Переваги неконтрольованого навчання:

- Не потребує маркованих даних
- Виявлення прихованих структур у даних
- Групування даних
- Виявлення аномалій

Недоліки неконтрольованого навчання:

- Складність інтерпретувати результати
- Чутливість до шуму в даних
- Складність вибору правильного алгоритму для рекомендаційної системи

1.2.2 Напівконтрольоване навчання

Напівконтрольоване навчання є посередником між контрольованим і неконтрольованим навчанням. Під час навчання він використовує менший набір даних з мітками, щоб керувати класифікацією та виділенням ознак із більшого набору даних без міток. Напівконтрольоване навчання може вирішити проблему відсутності достатньої кількості позначених даних для алгоритму контрольованого навчання [4].

Переваги напівконтрольованого навчання:

- Напівконтрольоване навчання може значно підвищити точність моделі порівняно з неконтрольованим навчанням, а також може бути більш ефективним, ніж контрольоване навчання, коли мічених даних мало.
- Напівконтрольоване навчання може допомогти зменшити потребу в мічених даних, використовуючи немічені дані для додаткового навчання моделі.

1.2.3 Навчання з підкріпленням

Головна ідея цього методу полягає в тому, що агент вчиться виконувати завдання через взаємодію з навколишнім середовищем, отримуючи винагороди або покарання за свої дії. Це схоже на те, як дитина вчиться ходити: пробує різні дії та отримуючи позитивні або негативні підказки від оточення.

Агент – це об'єкт або програма, яка приймає рішення, наприклад, дитина. Середовище – це все, з чим взаємодіє агент, для дитини це її дім, можливо садок. Дії – це різні можливі кроки, які може зробити агент. Стан – це інформація про поточний стан середовища, наприклад, голод, спрага, втома. Винагорода – це зворотній зв'язок від середовища на основі дій агента.

Процес навчання з підкріпленням проходить через цикл взаємодії агента із середовищем. Агент спостерігає поточний стан середовища, на основі цього стану вибирає дію. Середовище відповідає на цю дію, змінюючи свій стан та надаючи агенту винагороду або штраф. Агент оновлює свої знання на основі отриманої інформації та намагається обрати кращу дію у майбутньому [5].

Переваги навчання з підкріпленням включають можливість вчитися без необхідності ручного програмування всіх можливих сценаріїв та дій. Це особливо

корисно в складних та динамічних середовищах, таких як автономне керування автомобілями, ігри, робототехніка та управління ресурсами. Завдяки цьому підходу системи штучного інтелекту можуть ефективно вирішувати завдання, які раніше вважалися занадто складними або непередбачуваними для традиційних алгоритмів.

1.2.4 Контрольоване навчання

Контрольоване машинне навчання, характеризується використанням маркованих наборів даних для навчання алгоритмів класифікації даних або прогнозування результатів. Позначені набори даних складаються з входів (наприклад, різні геометричні фігури) кожен з яких має відповідний правильний вихід (мітки з назвами фігур). Завдання алгоритму полягає в тому, щоб навчитися передбачати правильні виходи для нових, невідомих входів, спираючись на інформацію, отриману з тренувального набору (див Рисунок 1.2) [6].

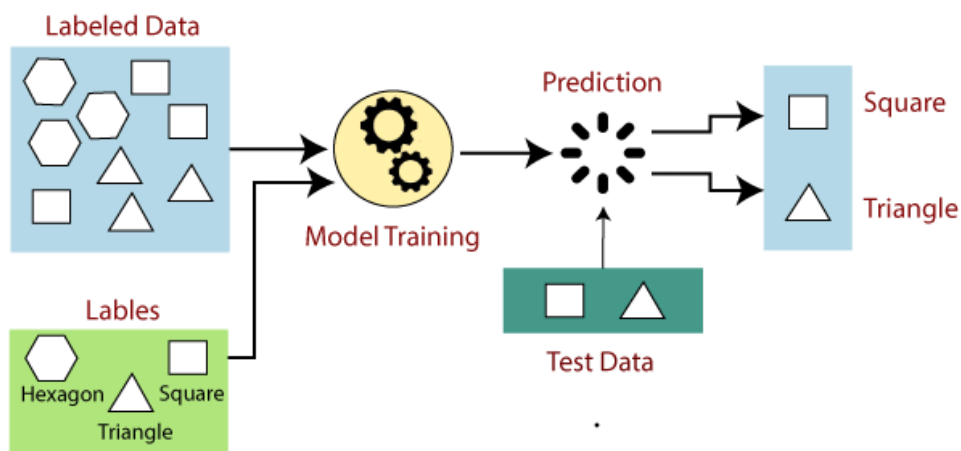


Рисунок 1.2 Приклад використання контрольованого навчання

Завдяки навчанню на позначених даних, моделі можуть досягати високої точності в прогнозуванні нових, невідомих даних. Наприклад, моделі для класифікації зображень здатні ідентифікувати об'єкти на зображеннях з точністю понад 90%. Це означає, що вони можуть відрізнити кішку від собаки або виявити дефекти на виробничій лінії з мінімальною кількістю помилок.

Саме через використання маркованих даних і високу точність прогнозування, я обрав неконтрольоване навчання для створення рекомендаційної системи.

Рекомендаційні системи виникли і почали розвиватися з середини 90-х років минулого століття. Це є чудовим прикладом того, як сучасні технології можуть допомогти у складних завданнях, таких як вибір фільму для перегляду чи музики для прослуховування. Вони використовують алгоритми контрольованого навчання для аналізу даних про користувачів та елементи контенту з метою рекомендації відповідного вмісту. В цьому процесі використовуються різні підходи, такі як спільна фільтрація (колаборативна фільтрація), фільтрація на основі вмісту, фільтрація на основі демографічних даних та інші [7].

1.2.4.1 Колаборативна фільтрація

Системи рекомендацій, що використовують алгоритми спільної фільтрації, є одним з найпоширеніших і найефективніших способів надавати користувачам персоналізовані рекомендації. Цей підхід базується на ідеї, що схожі користувачі будуть мати схожі уподобання, тому рекомендації для одного користувача можуть бути корисними іншим користувачам з подібними смаками.

При використанні алгоритмів спільної фільтрації, система рекомендацій аналізує дані про взаємодію користувачів з різними елементами контенту. Ця взаємодія може бути виражена у вигляді оцінок, переглядів або інших дій.

Одна з ключових переваг цього підходу полягає в тому, що він не потребує великої кількості додаткової інформації про користувачів, окрім їхньої взаємодії з системою. Наприклад, якщо два користувачі мають подібні смаки у фільмах і один з них вже оцінив певний фільм, система може рекомендувати цей фільм іншому користувачеві, оскільки ймовірно, що йому він також сподобається.

Таким чином, алгоритми спільної фільтрації є потужним інструментом для створення ефективних систем рекомендацій, які забезпечують персоналізований контент для кожного користувача [8-10].

Зараз розвиваються різні напрямки алгоритмів колаборативних фільтрацій. Ці алгоритми можуть бути класифіковані на три основні категорії, кожна з яких має свої особливості та переваги:

- **Memory-based**, є поширеним підходом до надання користувачам рекомендацій щодо товарів чи послуг. Він ґрунтується на ідеї, що схожі користувачі або схожі елементи мають подібну поведінку чи вподобання. Існує два основних підходи до створення таких систем: user-based та item-based.
 - **User-based**, рекомендує елементи користувачеві, базуючись на поведінці інших користувачів, які мають схожі вподобання. Алгоритм починається зі збору даних, аналізуючи історію дій користувача, такі як рейтинги. Потім для кожного користувача обчислюється схожість з іншими користувачами. Далі вибираються n користувачів, які мають найвищу схожість з цільовим користувачем. На основі дій цих n сусідів робляться прогнози щодо поведінки цільового користувача, наприклад, які фільми йому можуть сподобатися. Елементи, які отримали високі прогнози, рекомендуються цільовому користувачеві. Формула для розрахунку прогнозованого рейтингу $\hat{r}_{u,i}$ може бути записана так:

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N_i(u)} w_{uv}(r_{vi} - \bar{r}_v)}{\sum_{v \in N_i(u)} |w_{uv}|}, \quad (1.1)$$

де r_{vi} – оцінка від користувача v для фільму i

$\bar{r}_u$ – середнє з оцінок фільмів, користувача u

w_{uv} – міра схожості

$N_i(u)$ – множина фільмів, яку оцінив користувач u , які найбільш схожі до товару i

Детальніше про цей метод у підрозділі 3.2 Рекомендаційна система.

- **Item-based**, рекомендує нові елементи користувачеві, аналізуючи його взаємодії з подібними елементами. Спершу аналізуються взаємодії користувача з різними елементами, наприклад, оцінки фільмів. Для кожного елемента обчислюється схожість з іншими елементами, при цьому не враховуються характеристики самих елементів, а лише те, як користувачі взаємодіють з ними. Наприклад, якщо багато користувачів оцінили два фільми високо, ці фільми вважаються схожими. Далі робиться прогноз щодо того, як користувач оцінить нові елементи на основі його оцінок схожих елементів. Елементи, які отримали високі прогнози, рекомендуються цільовому користувачеві. Формула для розрахунку прогнозованого рейтингу $\hat{r}_{u,i}$, дуже схожа на формулу із попереднього підходу і може бути записана так:

$$\hat{r}_{ui} = \bar{r}_i + \frac{\sum_{v \in N_u(i)} w_{ij}(r_{uj} - \bar{r}_j)}{\sum_{v \in N_u(i)} |w_{ij}|}. \quad (1.2)$$

- **Model-based**, створює прогнозну модель машинного навчання даних. Модель використовує поточні значення в матриці елементів користувача як навчальний набір даних і створює прогнози для відсутніх значень за допомогою результуючої моделі. Таким чином, методи, засновані на моделі, використовують методи науки про дані та алгоритми машинного навчання, такі як дерева рішень, класифікатори Байєса та нейронні мережі, щоб рекомендувати елементи користувачам.

Матрична факторизація — це широко обговорюваний метод спільної фільтрації, який часто класифікують як тип моделі прихованих факторів. Будучи моделлю прихованого фактора, матрична факторизація передбачає, що подібність між користувачами або предметами можна визначити за допомогою вибраної кількості функцій. Наприклад, оцінку книги користувача можна передбачити, використовуючи лише жанр книги та вік або стать користувача. Основна формула для матричної факторизації виглядає так:

$$\hat{R} \approx P * Q^T. \quad (1.3)$$

Нехай $\hat{R}$ — це матриця оцінок користувачів розміром $m \times n$, де m — кількість користувачів, а n — кількість елементів. Матрична факторизація розкладає $\hat{R}$ на дві менші матриці P і Q , де

P — матриця користувачів розміром $m \times k$

Q — матриця елементів розміром $n \times k$

k — кількість прихованих факторів

Кожен елемент $\hat{r}_{ui}$ в матриці $\hat{R}$ обчислюється як скалярний добуток відповідних рядків матриць P та Q :

$$\hat{r}_{ui} = p_u * q_i^T, \quad (1.4)$$

де p_u — рядок u -го користувача з матриці P

q_i — рядок i -го елемента з матриці Q

- **Hybrid method**, поєднує в собі обидва попередні підходи. Це поширений метод при створенні рекомендаційних систем для комерційних сайтів. Гібридний метод об'єднує кілька стратегій рекомендацій, щоб використати їхні сильні сторони та компенсувати слабкі сторони кожного окремого підходу. Серед популярних підходів виділяють зважене об'єднання, коли рекомендації різних алгоритмів комбінуються шляхом присвоєння їм різних ваг; комбінування, коли система вибирає між різними підходами залежно від контексту або певних критеріїв; змішане представлення, коли рекомендації з різних алгоритмів представляються разом; каскадування, коли один алгоритм використовується для отримання початкового списку рекомендацій, а потім інший алгоритм уточнює цей список; Гібридний підхід допомагає вирішити проблему розрідженості даних і втрати інформації. Однак він складний та вимагає значних витрат на реалізацію та впровадження.

1.2.4.2 Фільтрація вмісту

Фільтрування вмісту є ще одним ефективним методом для створення рекомендаційних систем. У цьому підході рекомендації базуються на характеристиках або атрибутах елементів та уподобаннях користувача, які виражені через ці атрибути. Ключова особливість фільтрування вмісту полягає в тому, що воно не залежить від поведінки інших користувачів. Замість цього, воно спирається на аналіз характеристик елементів та створення профілю користувача на основі їхніх вподобань.

Наприклад, якщо система рекомендацій фільтрації вмісту помітить, що вам сподобалися такі фільми, як «Карти, гроші, два стволи» та «Великий куш», вона може запропонувати інший фільм із подібними жанрами та/або акторським складом, як-от «Джентльмени» [11].

Порівняно з фільтруванням вмісту, колаборативна фільтрація може бути більш персоналізованою і здатною до врахування складних взаємозв'язків між користувачами та елементами. Наприклад, якщо два користувачі мають схожий смак, але дивляться різні жанри фільмів, система колаборативної фільтрації може рекомендувати їм елементи, які сподобалися іншим користувачам зі схожим смаком, навіть якщо ці елементи належать до різних жанрів, що не могла б визначити фільтрація вмістом (див. Рисунок 1.3).

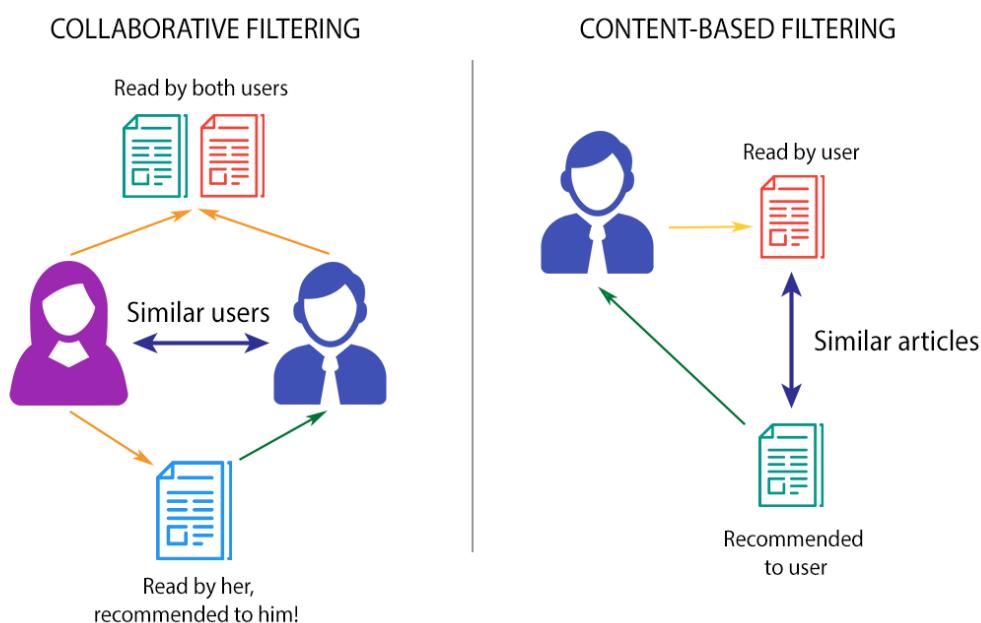


Рисунок 1.3 Різниця між колаборативною фільтрацією та фільтрацією за вмістом

Фільтрування вмісту можна розглядати як задачу регресії, де метою є прогнозування рейтингу, який користувач надасть певному елементу на основі його характеристик. У цьому контексті кожен елемент представлений своїми характеристиками, а користувачі — своїми вподобаннями. Мета полягає в тому, щоб побудувати модель, яка може передбачити рейтинг, який користувач надасть кожному елементу на основі його характеристик та вподобань. Одним з підходів до розв'язання цієї задачі є використання методів машинного навчання, зокрема алгоритмів регресії, таких як лінійна регресія, дерева рішень, випадковий ліс тощо. У цьому випадку модель навчається на основі тренувальних даних, де кожен зразок представляє собою пару (характеристики елемента, рейтинг користувача). Після навчання модель може передбачити рейтинг для нових елементів, використовуючи їх характеристики. Зважаючи на те, що ми розглядаємо фільтрування вмісту як задачу регресії, де метою є прогнозування рейтингу користувача для кожного елемента на основі його характеристик, можна використовувати різні моделі для навчання та прогнозування. Формули для оцінки точності моделей можуть включати такі метрики, як середня квадратична помилка (MSE), середня абсолютна помилка (MAE), середньоквадратична помилка (RMSE) та інші, які відображають відмінність між прогнозованими та фактичними рейтингами користувачів для елементів.

$$MSE = \frac{1}{n} \sum_{ui}^n (r_{ui} - \hat{r}_{ui})^2, \quad (1.5)$$

$$MAE = \frac{1}{n} \sum_{ui}^n |r_{ui} - \hat{r}_{ui}|, \quad (1.6)$$

$$RMSE = \frac{1}{n} \sum_{ui}^n \sqrt{(r_{ui} - \hat{r}_{ui})^2}, \quad (1.7)$$

де n – кількість спостережень

r_{ui} – фактичний рейтинг спостереження

$\hat{r}_{ui}$ – прогнозований рейтинг

1.2.4.3 Фільтрація контексту

Контекстне фільтрування додає інформацію про користувачів у процес рекомендацій. Один із прикладів використання цього підходу наводиться компанією Netflix на конференції NVIDIA GTC, де розповідалось про покращення рекомендацій шляхом уявлення рекомендації як прогнозування контекстної послідовності.

Цей підхід використовує послідовність дій користувача, а також поточний контекст, для прогнозування ймовірності наступної дії. У випадку Netflix, розглядається послідовність дій для кожного користувача - країна, пристрій, дата та час перегляду фільму. На основі цих даних навчається модель, щоб прогнозувати, що переглядатиме користувач у майбутньому.

Такий підхід дозволяє Netflix індивідуалізувати рекомендації на основі конкретного контексту кожного користувача. Наприклад, якщо користувач зазвичай дивиться фільми ввечері на телефоні в певній країні, Netflix може врахувати ці фактори для пропозиції відповідних фільмів на основі подібних виборів інших користувачів у подібних контекстах [12].

Висновки до Розділу 1.

Таким чином було проаналізовано чотири методи машинного навчання. У результаті аналізу можна зазначити наступне:

- **Неконтрольоване навчання** не відповідає даному проєкту через складність інтерпретації результатів. Цей підхід добре підходить для виявлення структур і закономірностей у невизначених даних, проте його результати часто важко пояснити і використати для конкретних рекомендаційних цілей.
- **Навчання з підкріпленням** також не підходить через ймовірний тривалий час навчання моделі та складність самого алгоритму. Цей метод ефективний для динамічних середовищ і може навчатися оптимальних дій через взаємодію з середовищем, але потребує багато часу та ресурсів для досягнення хороших результатів, що не підходить для даного проєкту.

- **Контрольоване навчання** виявилось найкращим вибором, оскільки воно використовується саме для маркованих наборів даних, які й будуть використовуватись у даному проєкті. Цей метод дозволяє створювати моделі, що можуть точно прогнозувати результати на основі відмічених даних, що є важливим для надання персоналізованих рекомендацій.

Після цього було проаналізовано наявні рекомендаційні алгоритми.

Серед них найкраще підходить **колаборативна фільтрація**, оскільки вона використовує уподобання схожих користувачів, що найкраще відображає інтереси користувача, який хоче отримати персоналізовані рекомендації. Колаборативна фільтрація має кілька переваг, які роблять її потужним інструментом для рекомендаційних систем: колаборативна фільтрація аналізує взаємодію користувачів з елементами та враховує їхні уподобання, щоб рекомендувати інші елементи, які можуть їм сподобатися; потребує мінімальну кількість інформації для створення рекомендацій; може виявляти складні зв'язки між користувачами та елементами, які можуть бути важко визначити за допомогою інших методів.

Алгоритм **фільтрації вмісту** виявився менш доречним, оскільки він просто шукатиме збіги за режисером фільму чи його жанром, що не завжди відображає справжні інтереси користувача. Цей метод добре працює для елементів, де можна чітко визначити відповідні атрибути, але не враховує загальні уподобання користувача.

Фільтрація контексту не була обрана через малу кількість інформації для її використання. Хоча цей метод може бути дуже ефективним, якщо є достатньо даних про контекст користувача, у даному випадку недостатня кількість контекстних даних обмежує його застосування.

Таким чином було обрано алгоритм **колаборативної фільтрації із підходом user-based**, як оптимальне рішення для реалізації персоналізованих рекомендацій у даному проєкті. Це дозволить максимально враховувати індивідуальні вподобання користувачів і надавати точні та релевантні рекомендації.

РОЗДІЛ 2. АНАЛІЗ ВИКОРИСТАНИХ ТЕХНОЛОГІЙ

Для вирішення поставленої задачі, було ухвалено розробити веб-застосунок, саме через його універсальність на різних пристроях, і легкість використання для користувачів.

2.1 Python-фреймворк Django

Django — це високорівневий веб-фреймворк Python, який допомагає з швидкою розробкою та практичним дизайном. Це перше про що подумаєш при розробці веб-додатку.

Веб-застосунки складаються з двох основних компонентів: серверної частини, яка містить серверний код, і клієнтської частини, що містить код, який виконується в браузері користувача. Коли користувач вводить URL-адресу в браузер, то відправляє запит на веб-сервер, на якому запущено веб-додаток. Веб-сервер обробляє цей запит з використанням серверного коду, взаємодіючи при необхідності з базою даних, і надсилає клієнту інформацію у вигляді html-сторінки. Код, який виконується на стороні клієнта, відображає цю інформацію користувачеві.

Веб-застосунки також можуть включати JavaScript для забезпечення динамічної взаємодії з користувачем без необхідності перезавантаження сторінки. Це дозволяє створювати інтерактивні та швидкі інтерфейси.

Django використовує архітектуру Model-View-Template (MVT), за допомогою якої, запитами та відповідями. Як можна зрозуміти ця архітектура поділяється на три частини: моделі (Model), представлення (View) та шаблони (Template) (див. Рисунок 2.1).

Model Django є посередником між базою даних і серверним кодом, забезпечуючи надійне джерело даних. Вони описують структуру даних через визначення полів і методів для взаємодії з базою даних. Моделі перетворюють таблиці бази даних у класи або об'єкти в коді Python.

View Django обробляють запити, отримуючи дані з моделей. Для кожного типу запиту, який користувач може надіслати, можна створити відповідну функцію

представлення. Ця функція отримує запит і повертає відповідь, яка може бути різного вигляду: код помилки, зображення, файл, тощо.

Template Django відповідає за візуальне представлення веб-сторінки у браузері. Більшість веб-сторінок написані мовою гіпертекстової розмітки (HTML), шаблони Django також використовують подібний синтаксис. Django підтримує власну мову шаблонів Django та популярну альтернативу під назвою Jinja2, але про це згодом.

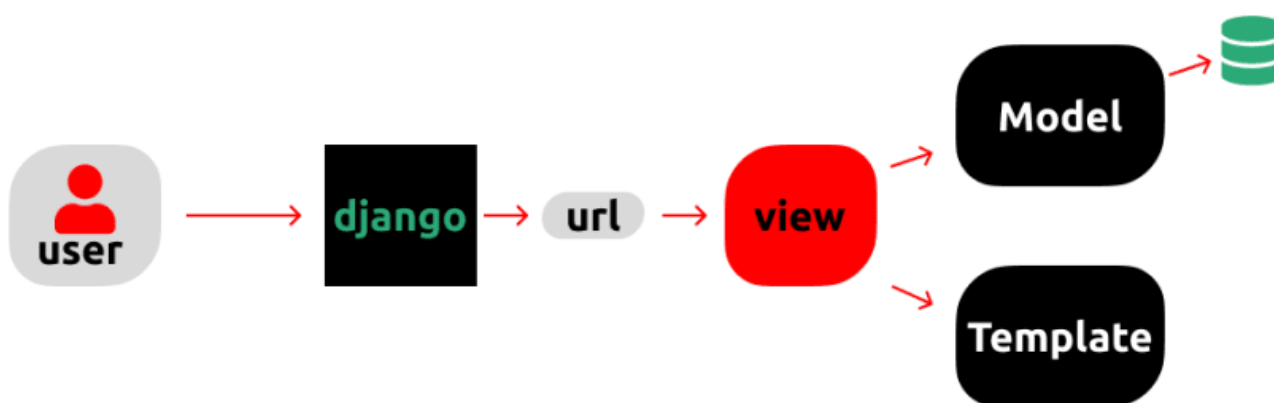


Рисунок 2.1 Взаємодія компонентів шаблону MVT один з одним для обслуговування запиту користувача.

Також важливо зазначити вибір саме фреймворку Django, а не іншої популярної альтернативи Flask.

Переваги Django над Flask

- **Комплексність та вбудовані функції**

Django має багато вбудованих інструментів та бібліотек для виконання різноманітних завдань, таких як аутентифікація користувачів, адміністрування, обробка форм, захист від атак типу CSRF (Cross-Site Request Forgery) та багато іншого. Також Django включає панель адміністратора, яка є веб-додатком, що забезпечує інтерфейс користувача для управління даними.

Flask, навпаки, є мікрофреймворком, що надає лише основний набір інструментів, дозволяючи вибір додаткових бібліотек та розширень.

- **Об'єктно-реляційне зіставлення (ORM)**

Django має вбудовану систему ORM, яка значно спрощує роботу з базою даних, дозволяючи розробникам маніпулювати даними за допомогою Python-класів та об'єктів замість написання SQL-запитів.

Flask не має вбудованого ORM, але підтримує інтеграцію з популярними сторонніми ORM, такими як SQLAlchemy.

- **Структура проєкту**

Django забезпечує чітко визначену структуру проєкту, що сприяє кращій організації коду та його підтримці.

Flask надає більше свободи в організації коду, що може бути корисно для невеликих проєктів або досвідчених розробників.

Оскільки основні функції що потрібні для даного проєкту вже доступні у Django, а у Flask усі ці функції стануть доступними за допомогою розширень та бібліотек, було прийнято рішення, що для розробки даного проєкту буде використовуватись Django [13].

2.2 Шаблонізатор

Як вже було згадано Django має вбудований шаблонізатор, проте також можна використовувати шаблонізатор Jinja2.

Jinja2 - це окремий рушій шаблонів, який сумісний з різними фреймворками, включаючи Django. У той час, як шаблони Django є стандартним вибором для програм Django, також можна використовувати Jinja2 з Django. Однак, згідно з опитуванням розробників Django у 2023 році, лише 16% з них це роблять [14].

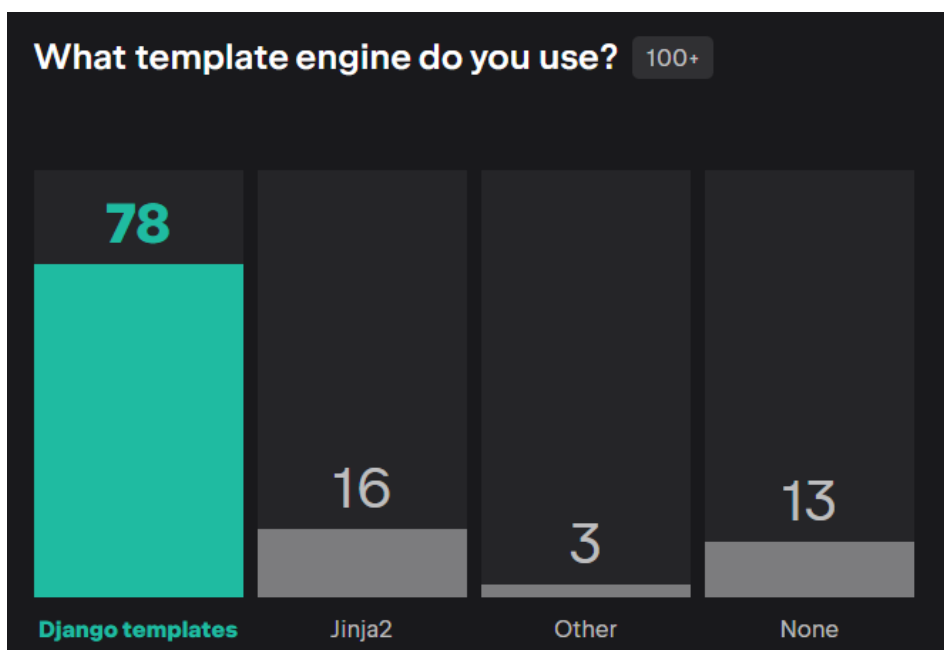


Рисунок 2.2 Опитування про використання шаблонізатора у Django

Jinja2 має більш складніший синтаксис, тоді коли шаблони Django менш гнучкі та обмежені. Для простих програм практично майже різниці, але можна зіткнутися з деякими обмеженнями, якщо потрібно виконувати складні операції у шаблонах проекту.

На Jinja2 сильно вплинула мова шаблонів Django, і тому їх синтаксис доволі схожий. Наприклад, обидва використовують подвійні фігурні дужки (`{{ }}`) для вставки змінних і фігурні дужки зі знаками відсотків (`{% %}`) для логічних конструкцій, таких як цикли та умовні оператори. Відрізняються лише деякими незначними функціями, що показано на Рисунку 2.3.



Рисунок 2.3 Порівняння вбудованого шаблонізатора Django та Jinja2

Оскільки визначних плюсів у додатковому встановлені Jinja2 немає, для розробки проєкту буде використовуватись вбудований шаблонізатор Django.

2.3 TMDb API

TMDb API – це безкоштовна API, призначена для отримання та використання даних із TMDb. Зазвичай це зображення або даних з фільмів, але також там є величезна кількість різноманітних запитів. Для того щоб використовувати цей API, для початку потрібно зареєструватись та подати заявку на отримання API-ключа.

Також важливо ознайомитись з умовами використання, оскільки там є декілька важливих пунктів:

- Комерційне використання API дозволене лише за письмовою згодою компанії
- Обов'язковим є використання логотипу компанії у веб-застосунку, щоб він був не менш помітним за власний логотип застосунку.
- Обов'язкове розміщення наступного тексту у помітному місці: This [website, program, service, application, product] uses TMDb and the TMDb APIs but is not endorsed, certified, or otherwise approved by TMDb. (укр. Цей [веб-сайт, програма, послуга, додаток, продукт] використовує TMDb та API TMDb, але не схвалений, сертифікований чи іншим чином підтверджений TMDb).

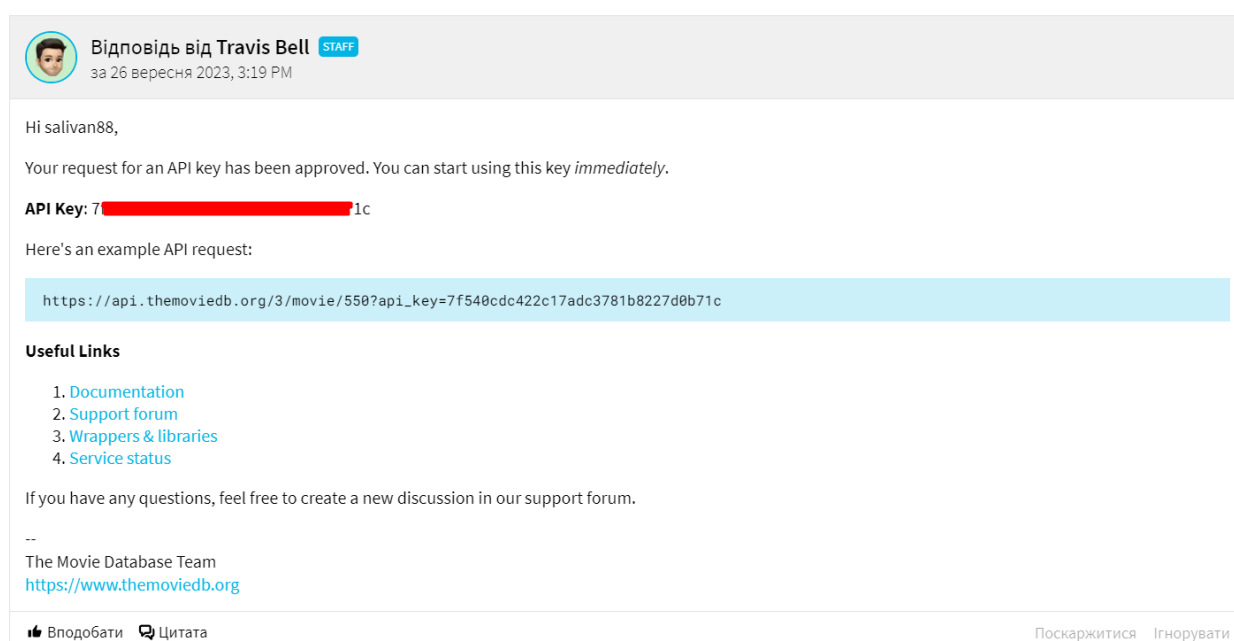


Рисунок 2.4 Лист з отриманням API-ключа

Отже після отримання API-ключа, можна ознайомитись з документацією, вибрати мову програмування серед наявних та одразу ж випробувати усі наявні запити та отримати на них відповідь.

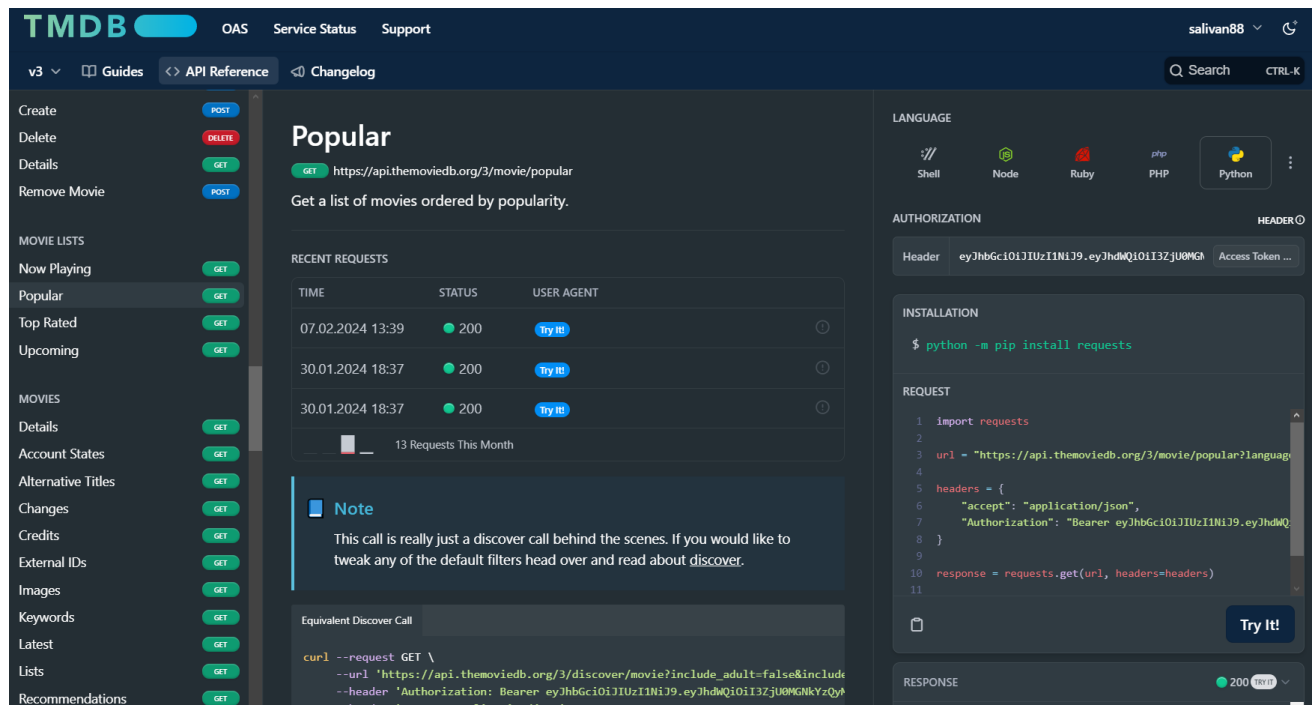


Рисунок 2.5 Вигляд документації для запиту "popular", що буде використано у даному проєкті

Як вже було згадано раніше використання TMDb API, значно полегшить розробку проєкту, оскільки не потрібно буде завантажувати велику кількість даних відеопродукцій у базу даних, що отримати їх відомості, а можна буде просто використовувати запити [15].

2.4 Система управління базами даних

Хоч вище вказано що використання бази даних значно зменшиться, але воно все ж таки буде, і для цього потрібно вибрати систему управління базами даних (СУБД). За замовчуванням у Django встановлено SQLite, вона є простою у використанні та підтримує більшість стандартних функцій.

Інші ж СУБД, такі як MySQL та PostgreSQL, призначені для більш обширного використання. Кожна з цих СУБД має свої переваги та обмеження, і вибір між ними залежить від конкретних потреб проєкту.

Оскільки у даному проєкті БД планується використовуватись лише для створення таблиці «ID_Користувача_ID_Фільму», і базових вимог для системи аутентифікацій, тому для даного проєкту буде більш доцільно використовувати SQLite, оскільки вона використовується для застосунків із низькорівневим навантаженням.

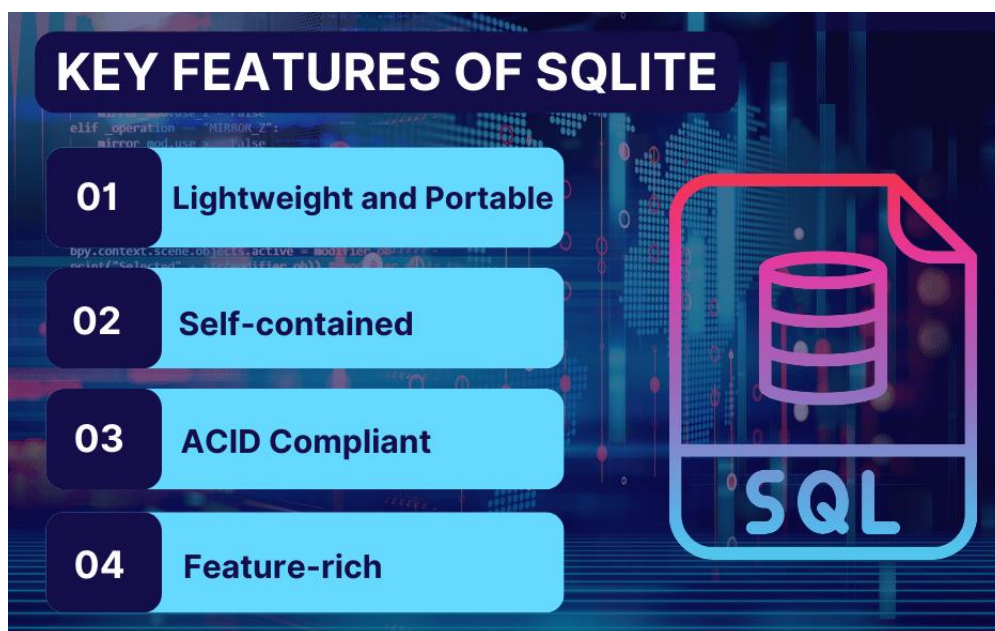


Рисунок 2.6 Основні характеристики SQLite

2.5 Git та GitHub

Для відслідковування та контролю змін у вихідному коді, проєкт було завантажено на сервіс GitHub. Це платформа для розробників, яка дозволяє створювати, зберігати, керувати та ділитися своїм кодом. Він використовує програмне забезпечення Git, забезпечуючи розподілений контроль версій, контроль доступу, відстеження помилок, запити функцій програмного забезпечення, керування завданнями, постійну інтеграцію. Операції в Git виконуються локально, що забезпечує високу швидкість роботи, а система оптимізована для обробки великих проєктів, що робить її популярною серед великих компаній та open-source проєктів.

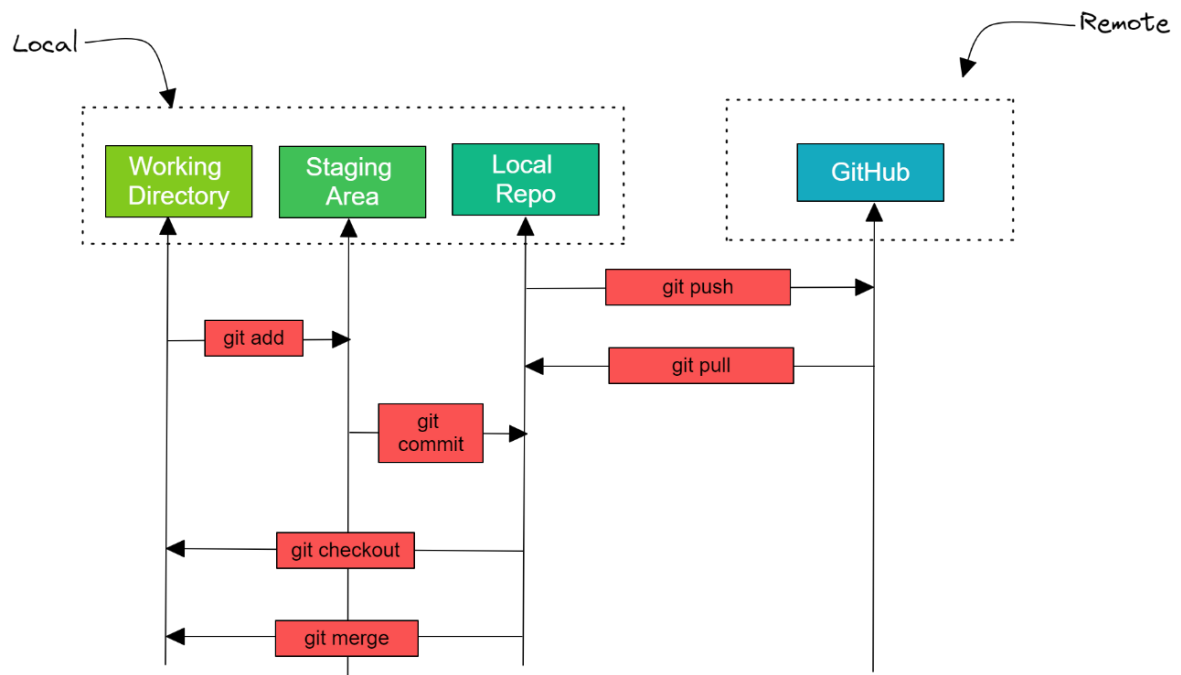


Рисунок 2.7 Найбільш популярні функції для контролю версій

Система контролю версій - це інструмент, який відстежує та фіксує всі зміни у файлах або групі файлів протягом певного періоду часу, щоб мати змогу повернутися до будь-якої попередньої версії у майбутньому [16, 17].

РОЗДІЛ 3. ОПИС РЕАЛІЗАЦІЇ ПРОЄКТУ

3.1 Серверна частина

Процес створення веб-додатку на Django починається з розробки серверу, де налаштовуються всі можливі посилання, представлення, база даних та інші необхідні компоненти. Оскільки сервер може містити у собі декілька додатків, для початку потрібно створити один. Далі у файлі `urls.py` потрібно налаштувати посилання на новий додаток, а у файлі `settings.py`, налаштувати посилання на статичну папку, у якій будуть зберігатися файли стилів `.css`, скриптів `.js` та зображення.

Після базового налаштування потрібно створити шаблони, які будуть відображати вміст сторінок. Вони зберігаються у папці `templates`.

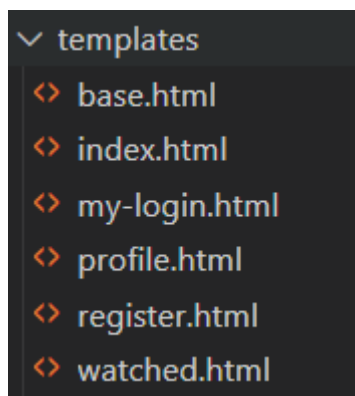


Рисунок 3.1 Структура папки `templates`

Після цього звісно ж для них потрібно створити посилання у файлі `urls.py` та представлення у файлі `views.py`.

Файл `urls.py` визначає, як URL-адреси додатку співвідносяться з конкретними функціями або класами обробників у `views.py`. Основна мета `urls.py` - маршрутизація запитів до відповідних обробників у додатку.

Файл `views.py` містить функції або класи, що визначають логіку обробки запитів та формування відповідей. Кожна функція або клас у `views.py` відповідає за обробку певного типу запитів та повернення відповідних відповідей, які можуть включати рендер HTML-шаблонів, повернення JSON-даних або перенаправлення користувачів.

Система аутентифікації в Django відповідає за управління процесами входу, реєстрації та перевірки користувачів на сайті. Вона виконуються в основному на сторінках `my-login.html` та `register.html`, а на інших просто підтягується за допомогою вбудованої змінної `{% if user.is_authenticated %}`.

Сама система аутентифікації це трохи розширена вбудована система Django. Налаштування її відбувається у файлі `forms.py`, який дозволяє налаштовувати моделі. Тому що дані користувачів зберігаються саме в БД, за що і відповідають моделі, але про це згодом.

Також у додатку є вбудована адміністративна панель. Адміни можуть бачити список усіх зареєстрованих користувачів, їхні профілі та основні деталі, такі як ім'я, електронна пошта, дата реєстрації, статус облікового запису тощо. Можливість редагування інформації про користувачів, включаючи зміни персональних даних, прав доступу та інших налаштувань. Адміністратори можуть призначати та змінювати ролі користувачів, надаючи їм різні рівні доступу до функціональності додатка [18].

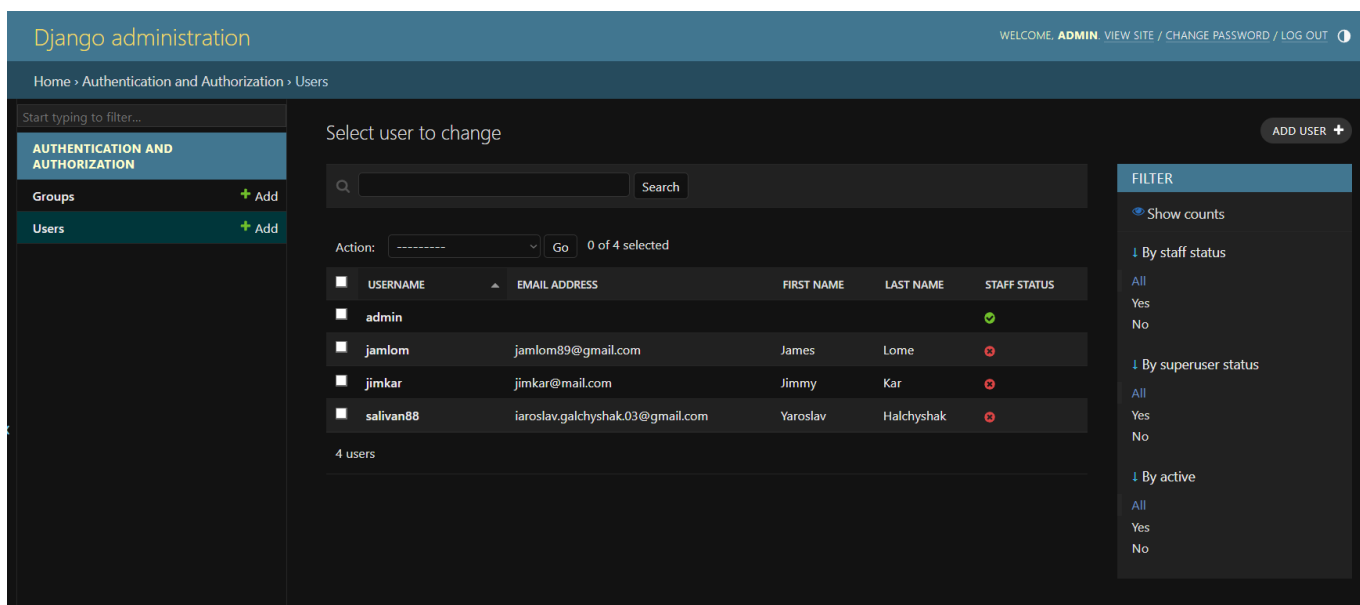


Рисунок 3.2 Вигляд адміністративної панелі

Для побудови рекомендаційної системи використовується лише одна таблиця бази даних (див. *Таблицю 3.1*), яка зберігає дані, необхідні для генерації рекомендацій. Створення нових таблиць та керування структурою бази даних у Django відбувається у файл `models.py`.

Таблиця 3.1 Структура таблиці «app_usermovie»

Ім'я	Тип	Опис
ID	Integer	Первинний ключ
User_id	Integer	ID користувача
Movie_id	Integer	ID фільму
Rating	Real	Рейтинг фільму

Проте, в базі даних також присутні вбудовані таблиці, що належать до системи аутентифікації, які відповідають за зберігання інформації про користувачів та їх облікові записи. На *Рисунку 3.3* можна побачити усі базові таблиці.

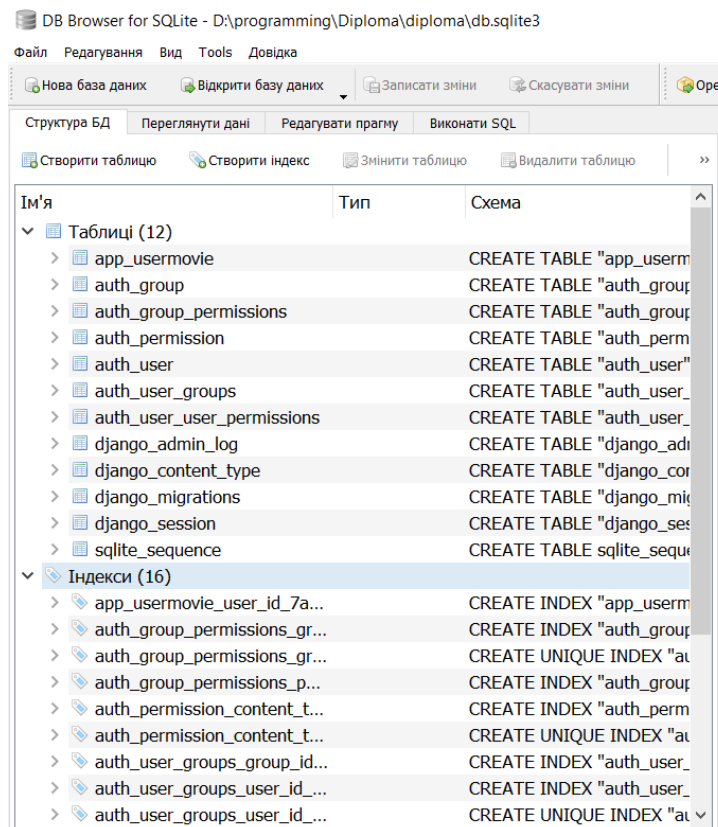


Рисунок 3.3 Загальний вигляд БД

У кінцевому варіанті структура даного проєкту буде виглядати як на *Рисунку 3.4*.

З поміж тих файлів що вже були розглянуті, залишились ще:

- **wsgi.py** використовується для розгортання Django-додатка на серверах, які підтримують WSGI. Він служить точкою входу для WSGI-сумісних веб-серверів, які запускають додаток.
- **asgi.py** аналогічний wsgi.py, але використовується для асинхронних серверів, що підтримують ASGI. Це дозволяє обробляти асинхронні запити, такі як WebSockets.
- **tests.py** містить тести для додатка. Django використовує фреймворк для тестування, вбудований у Python, що дозволяє створювати юніт-тести для перевірки роботи різних частин додатка.
- **apps.py** містить конфігурацію додатка. Він визначає клас конфігурації додатка, який успадковується від AppConfig і може бути використаний для налаштування параметрів додатка.
- **migrations** містить файли міграцій, які створюються Django для відстеження змін у моделях і синхронізації їх з базою даних. Кожна міграція представляє собою файл, який описує зміни, що повинні бути застосовані до бази даних.

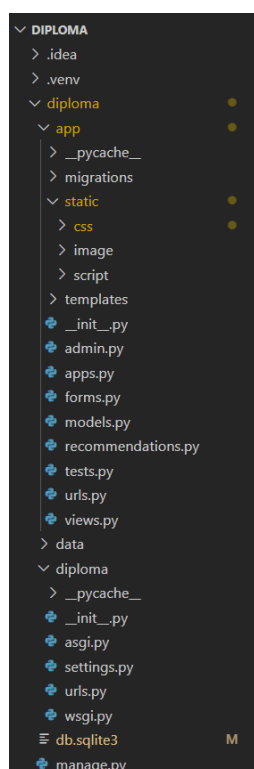


Рисунок 3.4 Загальна структура проєкту

Використання TMDb API також можна віднести до серверної частини. Інтеграція з TMDb API є важливою частиною серверної логіки даного проєкту. Вона дозволяє отримувати та обробляти інформацію про фільми, що значно спрощує функціональність додатка та покращує користувацький досвід. Завдяки простим запитам та гнучким параметрам, можна отримувати актуальну та детальну інформацію про популярні фільми та конкретні кінопроєкти. Приклад запиту що використовується:

`https://api.themoviedb.org/3/movie/${mode}?api_key=${apiKey}&page=1`

Цей запит містить змінну **mode**, яка отримує значення в залежності, який режим обрано «швидкий» чи «персональний», та змінну **apiKey**, що передає власний арі-токен. Він повертає JSON-об'єкт, що містить інформацію про фільми, включаючи назву, опис, дату виходу, рейтинг та інше.

```
{
  "page": 1,
  "results": [
    {
      "adult": false,
      "backdrop_path": "/fqv8v6AycXKsivp1T5yKtLb6Xce.jpg",
      "genre_ids": [
        878,
        12,
        28
      ],
      "id": 653346,
      "original_language": "en",
      "original_title": "Kingdom of the Planet of the Apes",
      "overview": "Several generations in the future following Caesar's reign, apes are now the dominant species and live harmoniously whi",
      "popularity": 5081.145,
      "poster_path": "/gKkl37BQuKTanygYQG1pyYglVgf.jpg",
      "release_date": "2024-05-08",
      "title": "Kingdom of the Planet of the Apes",
      "video": false,
      "vote_average": 7,
      "vote_count": 689
    },
  ],
}
```

Рисунок 3.5 Результат у вигляді JSON-об'єкту

Отримані дані обробляються на сервері та в клієнтському додатку. Необхідні поля з JSON-об'єкта витягуються та використовуються для подальшого відображення на стороні клієнта.

3.2 Рекомендаційна система

Для створення рекомендаційної системи за допомогою алгоритму колаборативної фільтрації із підходом user-based, спочатку потрібно знайти марковані дані. Вони будуть використовуватись для генерації персональних рекомендацій.

Для даного проєкту використовувався малий набір даних із сервісу MovieLens. У ньому наявні дані оцінювання фільмів від 600 різних користувачів [19].

userId,movieId,rating,timestamp	
1,1,4.0,964982703	
1,3,4.0,964981247	
1,6,4.0,964982224	
1,47,5.0,964983815	
1,50,5.0,964982931	
1,70,3.0,964982400	
1,101,5.0,964980868	
1,110,4.0,964982176	
1,151,5.0,964984041	
1,157,5.0,964984100	

Рисунок 3.6 Приклад файлу rating.csv

Як можна помітити на Рисунку 3.6, файл містить 4 колонки із даними userId, movieId, rating, timestamp, але movieId не збігається із ID фільмів у TMDb. Щоб не виникало проблем при відображенні рекомендацій, було використано ще один файл із архіву, а саме links.csv. Він мав відповідності movieId із ID фільмів у популярних БД.

movieId,imdbId,tmdbId	
1,0114709,862	
2,0113497,8844	
3,0113228,15602	
4,0114885,31357	
5,0113041,11862	
6,0113277,949	
7,0114319,11860	
8,0112302,45325	
9,0114576,9091	

Рисунок 3.7 Приклад файлу links.csv

Саме тому за допомогою Python-коду, було сформовано новий csv файл, де було замінено дані та прибрано колонку timestamp, так як вона є лишньою, а також значення ID користувачів було збільшено на 1000, щоб вони не пересікались із новими користувачами при створення матриці подібності.

Отже після зчитування даних із файлу csv та бази даних, ці дані об'єднуються у один список. Далі створюється матриця «Користувач-Фільм» (див. Рисунок 3.8), де рядки відповідають користувачам, стовпці - фільмам, а значення - це рейтинги. Матриця заповнюється нулями у тих випадках, коли користувач не оцінив фільм.

	11	12	13	14	15	18	22	24
1001	5.00000	0.00000	4.00000	5.00000	5.00000	0.00000	0.00000	0.00000
1002	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	4.00000
1003	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
1004	5.00000	0.00000	0.00000	5.00000	0.00000	0.00000	0.00000	0.00000
1005	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
1006	0.00000	0.00000	5.00000	0.00000	0.00000	0.00000	0.00000	0.00000
1007	5.00000	0.00000	5.00000	4.00000	0.00000	0.00000	4.50000	0.00000
1008	0.00000	0.00000	3.00000	0.00000	0.00000	0.00000	0.00000	0.00000
1009	0.00000	0.00000	0.00000	0.00000	5.00000	0.00000	0.00000	0.00000
1010	0.00000	3.50000	3.50000	1.00000	0.00000	0.00000	0.00000	0.00000
1011	0.00000	0.00000	5.00000	0.00000	0.00000	0.00000	0.00000	0.00000
1012	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
1013	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
1014	0.00000	0.00000	4.00000	0.00000	0.00000	0.00000	0.00000	0.00000
1015	5.00000	3.00000	5.00000	4.00000	0.00000	3.00000	0.00000	3.00000
1016	3.00000	0.00000	3.50000	3.50000	3.50000	0.00000	0.00000	0.00000
1017	5.00000	0.00000	5.00000	4.00000	0.00000	0.00000	0.00000	0.00000
1018	4.00000	4.00000	4.50000	0.00000	5.00000	3.50000	4.50000	3.50000
1019	4.00000	0.00000	2.00000	4.00000	0.00000	5.00000	0.00000	0.00000
1020	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000

Рисунок 3.8 Приклад матриці "Користувач-Фільм"

Далі обчислюється схожість між користувачами, використовуючи косинусну схожість, і повертає матрицю схожості користувачів (див. Рисунок 3.9). Косинусна схожість є метрикою, яка використовується для вимірювання схожості між двома векторами. Основна ідея косинусної схожості полягає в обчисленні косинуса кута між двома векторами в багатовимірному просторі. Чим менший кут між векторами, тим більше вони схожі. Вектори, що мають косинусну схожість, близьку до 1, майже співпадають, а ті що мають близьку до 0, майже ортогональні [20].

Формула для обчислення косинусної схожості між двома векторами виглядає наступним чином:

$$w_{uv} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}, \quad (3.1)$$

де

$A*B$ – це скалярний добуток векторів, який обчислюється:

$$A * B = \sum_{i=1}^n A_i B_i, \quad (3.2)$$

$\|A\|$ – це евклідова норма вектора, яка обчислюється як квадратний корінь із суми квадратів його компонент:

$$\|A\| = \sqrt{\sum_{i=1}^n A_i^2}, \quad (3.3)$$

	12	13	14	15	16	17	18
0	0.09297	0.11324	0.16069	0.16986	0.26436	0.21487	0.32538
1	0.04392	0.01690	0.11978	0.09373	0.10375	0.16625	0.01257
2	0.00000	0.00316	0.01782	0.03337	0.01014	0.02917	0.01977
3	0.07695	0.04899	0.07155	0.16476	0.14506	0.12322	0.20605
4	0.01716	0.22171	0.11015	0.08217	0.16263	0.12131	0.09876
5	0.02392	0.34644	0.06758	0.04927	0.09068	0.11558	0.22185
6	0.03208	0.07474	0.23048	0.15879	0.24650	0.27283	0.12697
7	0.05442	0.43006	0.10282	0.11472	0.20000	0.15217	0.16918
8	0.05022	0.00000	0.12708	0.07923	0.08981	0.11067	0.04887
9	0.03815	0.02719	0.10825	0.09819	0.07236	0.11514	0.02680
10	0.03058	0.20790	0.09025	0.05384	0.17109	0.15030	0.13960
11	0.03785	0.01942	0.04265	0.00000	0.02698	0.01359	0.07366
12	1.00000	0.06593	0.10674	0.08624	0.07398	0.07973	0.10644
13	0.06593	1.00000	0.07680	0.08359	0.16732	0.11861	0.08992
14	0.10674	0.07680	1.00000	0.23167	0.30507	0.30697	0.15777
15	0.08624	0.08359	0.23167	1.00000	0.45610	0.30365	0.15049
16	0.07398	0.16732	0.30507	0.45610	1.00000	0.32121	0.17819
17	0.07973	0.11861	0.30697	0.30365	0.32121	1.00000	0.18932
18	0.10644	0.08992	0.15777	0.15049	0.17819	0.18932	1.00000

Рисунок 3.9 Приклад матриці подібності

Після цього відбувається генерація рекомендації фільмів для користувача на основі схожості з іншими користувачами. Спочатку вона визначає індекс користувача в матриці рейтингів, що дозволяє ідентифікувати його вподобання. Потім функція знаходить усіх схожих користувачів, використовуючи матрицю схожості. Після визначення схожих користувачів, вони сортуються за ступенем схожості у порядку спадання, виключаючи самого користувача, оскільки він завжди буде найбільш схожим на себе. Далі вибираються n користувачів, які мають найвищу схожість з цільовим користувачем. На основі дій цих сусідів, обраховується рейтинг потенційних рекомендацій для користувача, тобто які фільми йому можуть сподобатися. Елементи, які отримали 5 найвищих рейтингів, рекомендуються користувачеві (див. Рисунок 3.10). Обрахунок відбувається за формулою 1.1 наведеною у розділі 1.2.4.1 *Колаборативна фільтрація*, у підпункті про user-based.

```
recommended_movies = {list: 1550} [(318, 47.48148148148149)
> 0000 = {tuple: 2} (318, 47.48148148148149)
> 0001 = {tuple: 2} (588, 39.98148148148149)
> 0002 = {tuple: 2} (589, 36.98148148148148)
> 0003 = {tuple: 2} (4993, 34.981481481481495)
> 0004 = {tuple: 2} (5952, 34.481481481481495)
> 0005 = {tuple: 2} (2762, 33.48148148148149)
> 0006 = {tuple: 2} (150, 33.48148148148148)
> 0007 = {tuple: 2} (364, 32.48148148148148)
> 0008 = {tuple: 2} (595, 29.98148148148148)
> 0009 = {tuple: 2} (58559, 29.481481481481485)
> 0010 = {tuple: 2} (34, 28.98148148148148)
> 0011 = {tuple: 2} (380, 28.98148148148148)
> 0012 = {tuple: 2} (597, 27.98148148148148)
> 0013 = {tuple: 2} (293, 27.481481481481488)
> 0014 = {tuple: 2} (377, 26.981481481481477)
> 0015 = {tuple: 2} (904, 26.481481481481488)
> 0016 = {tuple: 2} (32, 26.48148148148148)
> 0017 = {tuple: 2} (7153, 25.981481481481485)
```

Рисунок 3.10 Посортовані фільми за спаданням рейтингу

3.3 Користувацька частина

До користувацької частини можна віднести вигляд зі сторони користувача та його можливі дії. Як можна було помітити по структурі проєкту, усього буде три основні сторінки: «головна», «профіль користувача» та «список переглянутих фільмів». Також є додаткові сторінки, які належать до системи аутентифікації: «реєстрація», «логін». Сторінка «base.html» відповідає за відображення базових елементів, таких як хедер та футер. Хедер містить навігаційне меню, яке дозволяє користувачам переходити між основними сторінками сайту, тоді як футер включає додаткову інформацію. Головна сторінка включає у собі привітання та вибір режиму рекомендацій: «Швидкий» чи «Персональний».

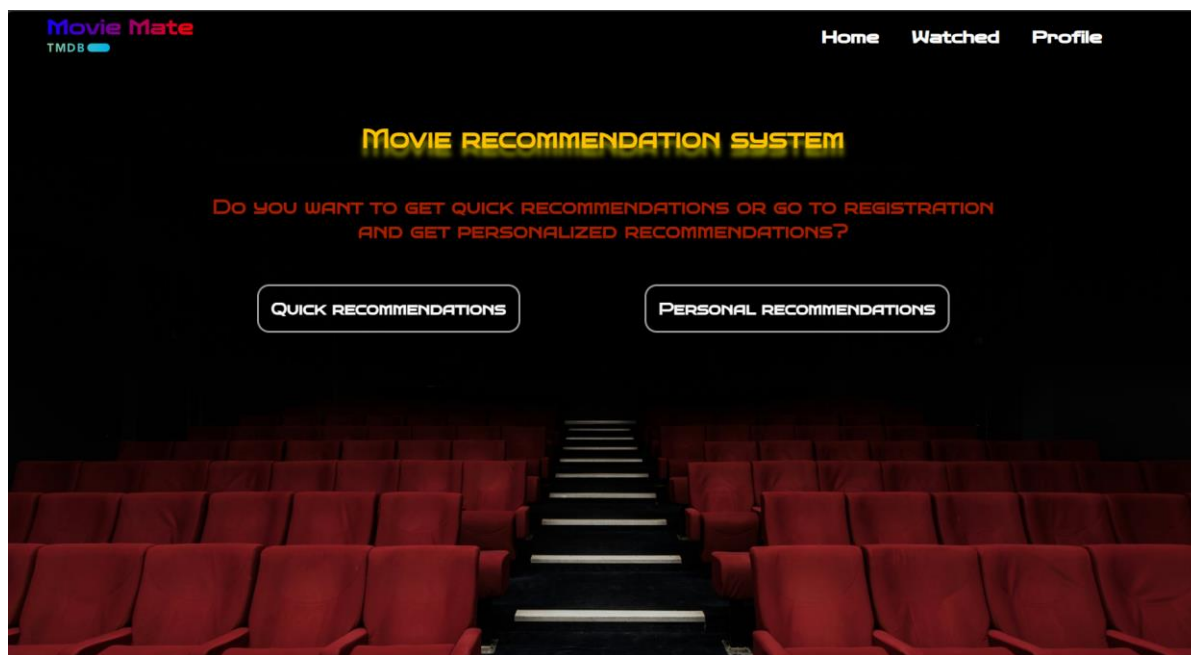


Рисунок 3.11 Вигляд "головної" сторінки

У «Швидкому» режимі користувач отримує загальні рекомендації. Тобто за допомогою API-запиту, із відповіді витягується детальна інформація про 20 популярних фільмів на даний момент. Серед яких він може вибрати які йому сподобались.

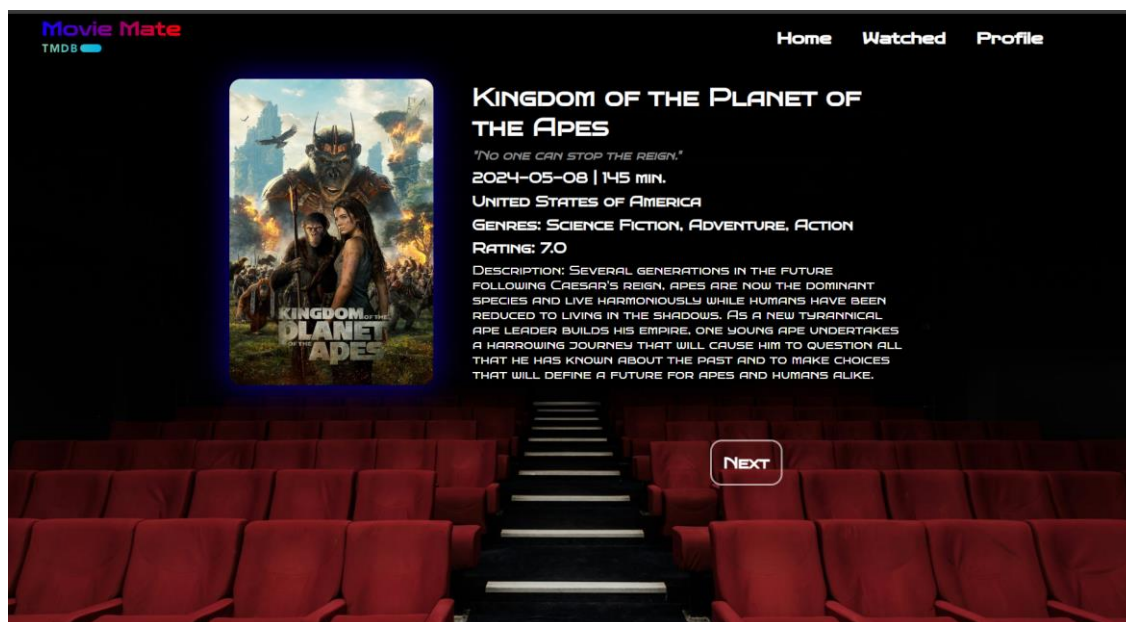


Рисунок 3.12 Режим "швидких" рекомендацій

Усе це відбувається на головній сторінці та динамічно оновлюється без перезавантажень за допомогою JavaScript. Тобто при виборі режиму рекомендацій, JavaScript відправляє запит на сервер, і в залежності від режиму, виконується певна частина скрипту, та відображається на екрані користувача. Після закінчення списку із популярними фільмами, користувачу пропонується зареєструватись або увійти в аккаунт.

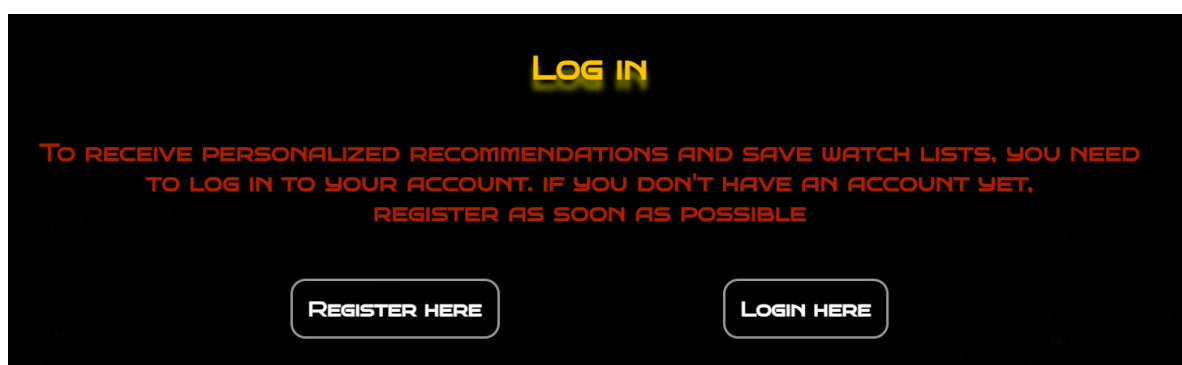


Рисунок 3.13 Спроба авторизації

Для того щоб зареєструватись, користувачу потрібно ввести своє ім'я, прізвище, нікнейм, е-пошту, придумати складний пароль, та повторити його. Також важливо зазначити політика аутентифікації Django включає кілька вимог до пароля,

щоб забезпечити його складність і унікальність. Зокрема, пароль не може складатися зі слів, схожих на ім'я, прізвище або електронну пошту користувача. Це допомагає запобігти використанню легко-передбачуваних паролів, які можуть бути зламані.

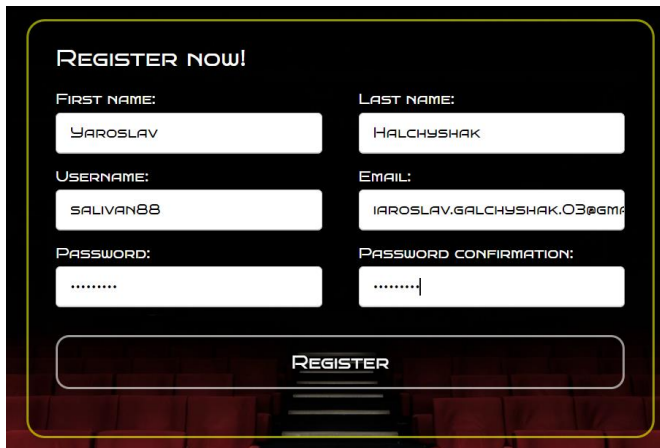
A registration form titled "REGISTER NOW!" with a dark background and a yellow border. It contains six input fields: "FIRST NAME:" with "YAROSLAV", "LAST NAME:" with "HALCHYSHAK", "USERNAME:" with "SALIVAN88", "EMAIL:" with "IAROSLAV.GALCHYSHAK.O3@gmail.com", "PASSWORD:" with masked characters, and "PASSWORD CONFIRMATION:" with masked characters. A "REGISTER" button is at the bottom.

Рисунок 3.14 Вигляд сторінки "Реєстрація"

Для входу у аккаунт потрібно просто ввести свій нікнейм та пароль, після успішної авторизації, користувача перекидає на його персональну сторінку, та у з'являється доступ до персоналізованих рекомендацій та збереження переглянутих фільмів.

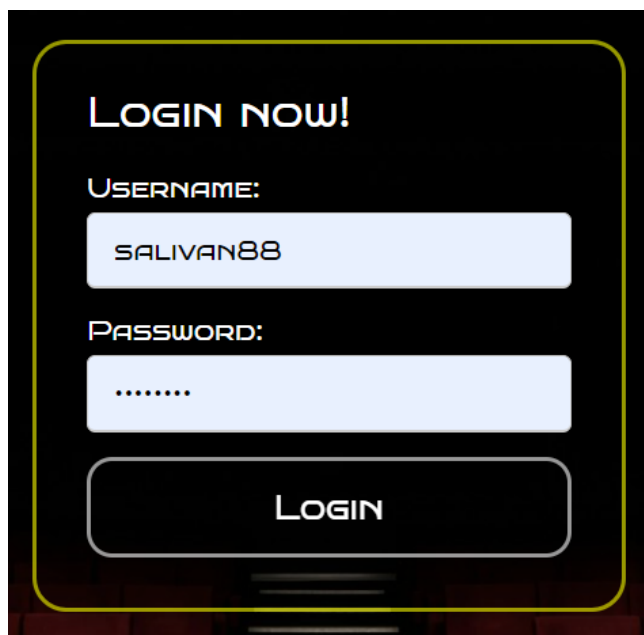
A login form titled "LOGIN NOW!" with a dark background and a yellow border. It contains two input fields: "USERNAME:" with "SALIVAN88" and "PASSWORD:" with masked characters. A "LOGIN" button is at the bottom.

Рисунок 3.15 Вигляд сторінки "Вхід у аккаунт"

Коли користувач тільки зареєструвався у його профілі є лише привітання, дата реєстрації та можливість виходу із аккаунту. Вже після отримання персональних рекомендацій на сторінці профілю з'явиться п'ятірка рекомендованих до перегляду фільмів.



Рисунок 3.16 Початковий вигляд сторінки "Профіль"

Отже перейдемо до режиму «Персональний», тут на допомозі є API-запит із найкращими фільмами, їх тут також 20, і також усі динамічно оновлюються за допомогою JavaScript. Основною відмінністю, окрім зміни API-запиту, є функція додавання рейтингу до переглянутих фільмів, на основі який будуть генеруватись рекомендації за допомогою машинного навчання.

Коли користувач ставить рейтинг фільму та натискає кнопку "Watched", починається процес відправки форми. Спочатку стандартна дія відправки форми блокується, щоб обробити її асинхронно, без перезавантаження сторінки.



Рисунок 3.17 Режим "Персональних" рекомендацій

Потім система визначає ID поточного фільму зі списку популярних фільмів, який відображається на сторінці, і цей ID записується у приховане поле форми, щоб ця інформація була відправлена на сервер разом з іншими даними форми.

Всі дані, які користувач ввів у форму, збираються в спеціальний об'єкт, включаючи рейтинг фільму. Якщо користувач не ввів рейтинг, система видає повідомлення про необхідність вибору рейтингу, і подальші дії не виконуються до тих пір, поки рейтинг не буде вказано. Після цього дані форми відправляються на сервер для обробки та записується у базу даних. Відправка відбувається асинхронно, що означає, що користувачеві не потрібно чекати перезавантаження сторінки.

Якщо сервер успішно отримує і обробляє дані, користувач отримує повідомлення про успішне додавання фільму до списку переглянутих. Рейтинг у формі скидається, щоб уникнути помилок при наступних відправках, а кнопка "Watched" стає недоступною (відключається), щоб користувач не зміг випадково додати той самий фільм вдруге. В результаті користувач бачить повідомлення (див. Рисунок 3.18), яке підтверджує, що фільм успішно доданий до списку переглянутих та може переходити далі. Вся взаємодія відбувається на тій самій сторінці без необхідності її перезавантаження, що забезпечує зручність і швидкість використання.



Рисунок 3.18 Повідомлення про успішне додавання фільму

Вибравши переглянуті фільми із 20 найкращих, що інформацію можна перевірити відкривши базу даних. Як можна було помітити мій ID – 12, та я переглянув лише 9 фільмів із 20. З Рисунку 3.19 можна помітити що дані успішно записались у базу даних та готові до створення рекомендацій.

Таблиця: app_usermovie

	id	user_id	rating	movie_id
	Філ...	Фільтр	Фільтр	Фільтр
20	71	11	2.0	429
21	134	12	5.0	278
22	135	12	4.0	129
23	136	12	4.0	155
24	137	12	5.0	496243
25	138	12	4.0	680
26	139	12	4.0	122
27	140	12	5.0	13
28	141	12	3.0	429
29	142	12	4.0	769

Рисунок 3.19 Оновлена таблиця у БД

Після проходження 20-го фільму, перед користувачем з'являється повідомлення «щоб отримати рекомендації потрібно перейти на сторінку власного профілю» та кнопка «Continue» .

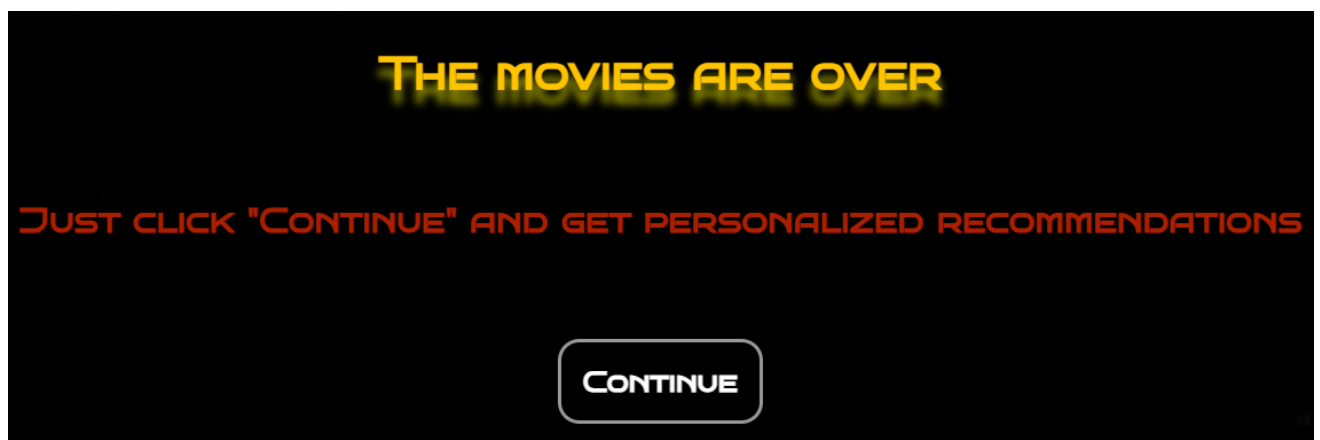


Рисунок 3.20 "За крок до отримання персональних рекомендацій"

Одразу після переходу на сторінці профілю користувача, з'являється п'ятірка рекомендованих фільмів. Рекомендовані фільми представлені у вигляді постерів та назв фільмів. Ці рекомендації ґрунтуються на вподобаннях користувача та його історії переглядів.

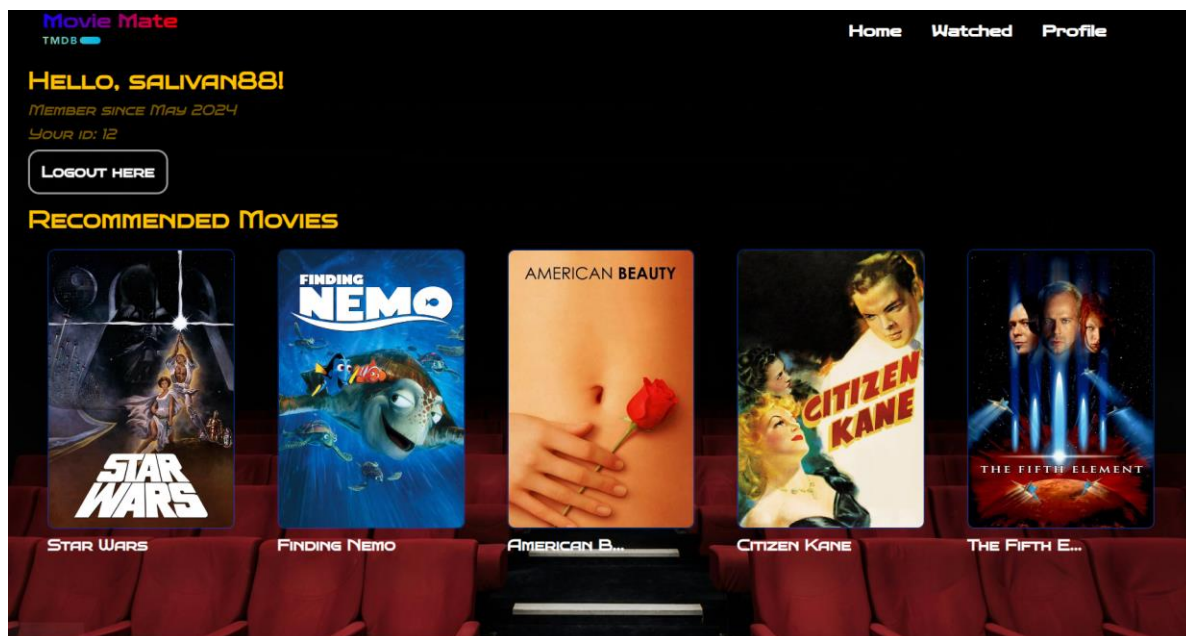


Рисунок 3.21 Результат персональних рекомендацій

Також варто сказати що вміст сторінки «Watched» оновився, тепер там відображаються фільми що були вибрані як переглянуті та занесені у базу даних.

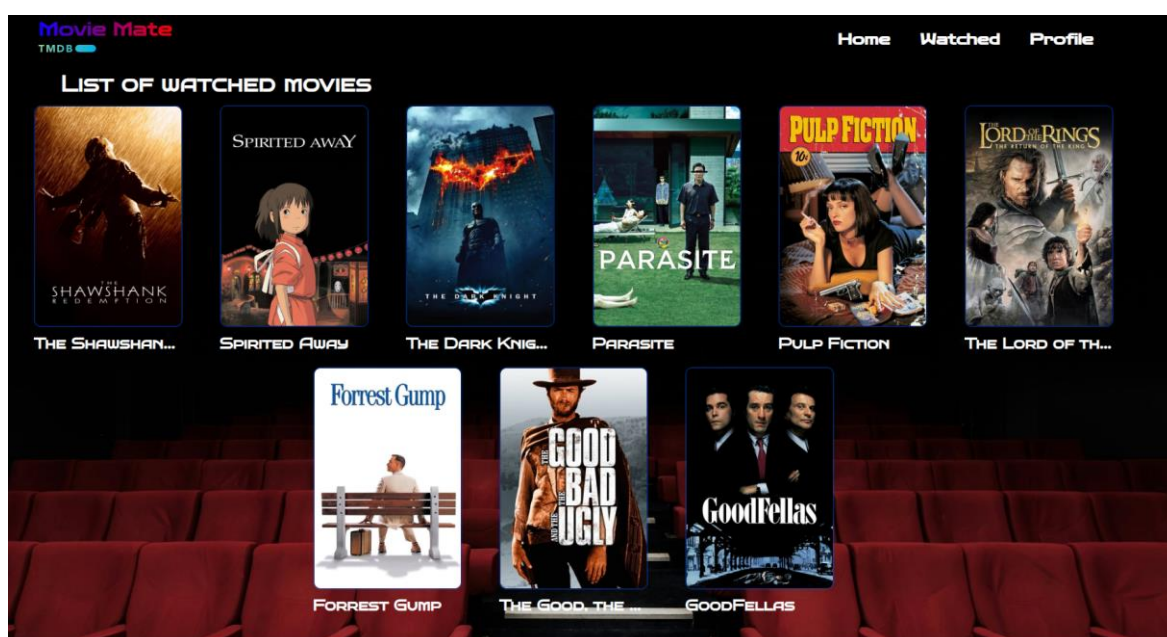


Рисунок 3.22 Вигляд сторінки "Watched"

3.4 Майбутній розвиток проєкту

Насправді проєкт має великі перспективи для подальшого розвитку, оскільки це сфера постійно розвивається та завжди буде актуальною. Перспективи можуть вивести його на новий рівень, зробивши ще зручнішим і кориснішим для користувачів.

Основним напрямком звісно ж є покращення алгоритмів рекомендаційної системи. Використання більш складних моделей машинного навчання, таких як глибокі нейронні мережі, може підвищити точність рекомендацій, що дозволить краще враховувати індивідуальні вподобання користувачів і адаптуватися до їхніх змін. Також можливість розширення, насамперед додати контентні та гібридні методи рекомендацій. Це означає, що система зможе враховувати не тільки історію переглядів користувача, але й такі фактори, як жанр, режисер, актори тощо, що забезпечить більш різноманітні і точні рекомендації.

Аналітика переглядів і відстеження трендів також можуть суттєво покращити сервіс. Збір та аналіз даних про перегляди фільмів дозволить покращити рекомендаційні алгоритми та маркетингові стратегії,

Важливим покращенням також можуть стати соціальні аспекти. Наприклад, впровадження системи друзів та форумів для обговорення, що дозволить користувачам додавати один одного в друзі, ділитися рекомендаціями і бачити, що їх друзі переглядали або рекомендують. Можливість залишати рецензії та коментарі до фільмів, а також обговорювати їх на форумах, зробить платформу більш живою і різноманітною.

Можливим великим припливом користувачів є розробка мобільного додатку. Це дозволить користувачам отримувати рекомендації в будь-який час і в будь-якому місці, що підвищить доступність сервісу.

Ще один перспективний напрямок – це покращення UI/UX (інтерфейсу користувача та користувацького досвіду). Поліпшення UI/UX може значно підвищити зручність і привабливість платформи для користувачів. Також можна зробити інтерфейс більш інтуїтивно зрозумілим і зручним у використанні. Це може

включати спрощення навігації, поліпшення розташування елементів на екрані та використання привабливих візуальних ефектів.

Усі ці напрямки розвитку сприятимуть покращенню користувацького досвіду, підвищенню задоволеності користувачів та зростанню популярності платформи. Проєкт зможе надавати користувачам ще більше можливостей.

ВИСНОВКИ

Дана робота є важливим кроком у напрямку покращення користувацького досвіду в сфері розваг. Успішна реалізація рекомендаційної системи дозволила створити сервіс, що надає користувачам персональні рекомендації на основі їхніх переглядів і вподобань, використовуючи сучасні алгоритми машинного навчання.

Основними досягненнями проєкту стали:

- Впровадження API-запитів для отримання актуальних даних про популярні фільми дозволило значно полегшити та розширити функціональність платформи та підвищити її актуальність
- Вдалося об'єднати користувацькі рейтинги та історію переглядів з існуючими даними з популярного відео-сервісу, що забезпечило актуальні дані для створення персональних рекомендацій
- Метод косинусної схожості, що використовувався у колаборативній фільтрації дозволив точно визначати ступінь схожості між користувачами та їхніми вподобаннями, що стало основою для персональних рекомендацій
- Алгоритм колаборативної фільтрації оснований на user-based підході, який допоміг точно прогнозувати рейтинги для рекомендованих фільмів
- Реалізація гостьової та авторизованої сесії забезпечила краще ознайомлення із сервісом, що дало користувачам можливість вибирати найзручніший для них варіант отримання рекомендацій

Таким чином, даний проєкт досягнув своїх основних цілей, створивши рекомендаційну систему, яка здатна надавати користувачам якісні та персональні рекомендації. Подальший розвиток проєкту має великий потенціал для покращення сервісу та залучення нових користувачів.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. The Best Video Streaming Services for 2024. URL: <https://www.pcmag.com/picks/the-best-video-streaming-services> (дата звернення: 23.05.24)
2. What Is Machine Learning (ML)?. URL: <https://ischoolonline.berkeley.edu/blog/what-is-machine-learning/> (дата звернення: 12.01.24)
3. Pro's and con's of unsupervised. URL: <https://eyer.ai/blog/pros-and-cons-of-supervised-vs-unsupervised-algorithms-for-scalable-anomaly-detection/> (дата звернення: 12.01.24)
4. Semi-Supervised Learning in ML. URL: <https://www.geeksforgeeks.org/ml-semi-supervised-learning/> (дата звернення: 12.01.24)
5. Richard S. Sutton, Andrew G. Barto. Reinforcement Learning: An Introduction. The MIT Press. 2014. 2-9 с.
6. Supervised Learning. URL: <https://www.ibm.com/topics/supervised-learning> (дата звернення: 13.01.24)
7. Recommender System Using Machine Learning. URL: <https://maddevs.io/blog/recommender-system-using-machine-learning/> (дата звернення: 13.01.24)
8. Лобур М. В., Шварц М. Є., Стех Ю. В. Моделі і методи прогнозування рекомендацій для колаборативних рекомендаційних систем. НУ «ЛП». 2018. 68-75 с.
9. G. Adomavicius, A. Tuzhilin. Toward the Next Generation of Recommender Systems. IEEE. 2005. 734-741 с.
10. J. Bobadilla, F. Ortega, A. Hernando, A. Gutiérrez. Recommender systems survey. Universidad Politecnica de Madrid. 2013. 109-114 с.
11. Introduction to Content Based Recommendation. URL: <https://redfield.ai/content-based-recommendation/> (дата звернення: 14.01.24)
12. Deep Learning for Recommender Systems. URL: <https://www.nvidia.com/en-us/on-demand/session/gtcsiliconvalley2018-s81011/> (дата звернення: 15.01.24)

13. Flask vs Django: Let's Choose Your Next Python Framework. URL: <https://kinsta.com/blog/flask-vs-django/> (дата звернення: 20.01.24)
14. Django Developers Survey 2023. URL: <https://lp.jetbrains.com/django-developer-survey-2023/> (дата звернення: 20.01.24)
15. Documentation of the TMDb API. URL: <https://developer.themoviedb.org/docs/getting-started> (дата звернення: 12.01.24)
16. Офіційний сайт «GitHub». URL: <https://github.com> (дата звернення: 24.01.24)
17. Офіційний сайт «Git». URL: <https://git-scm.com> (дата звернення: 24.01.24)
18. Documentation of the Django admin site. URL: <https://docs.djangoproject.com/en/4.2/ref/contrib/admin/> (дата звернення: 15.05.24)
19. MovieLens Latest Datasets. URL: <https://grouplens.org/datasets/movielens/latest/> (дата звернення: 05.04.24)
20. Косинус подібності. URL: https://uk.wikipedia.org/wiki/Косинус_подібності (дата звернення: 24.05.24)

ДОДАТОК А

Код програми

views.py

```
def index(request):
    return render(request, 'index.html')

def profile(request):
    return render(request, 'profile.html')

def watched(request):
    user_id = request.user.id
    user_movies = UserMovie.objects.filter(user_id=user_id)
    user_movies_json = serialize('json', user_movies)

    return render(request, 'watched.html', {'user_movies_json': user_movies_json})

def rec(request):
    user_id = request.user.id
    recommendations = recommend_movies_for_user(user_id)
    api_key = '7f540cdc422c17adc3781b8227d0b71c'

    movies = []
    for movie_id in recommendations:
        movie_details = get_movie_details(movie_id, api_key)
        if movie_details:
            movies.append(movie_details)

    context = {
        'movies': movies
    }
    return render(request, 'profile.html', context)

def get_movie_details(movie_id, api_key):
    url =
f"https://api.themoviedb.org/3/movie/{movie_id}?api_key={api_key}&language=en-US"
    response = requests.get(url)
    if response.status_code == 200:
        return response.json()
    return None

def register(request):
    if request.method == "POST":
        form = CreateUserForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect("my-login")
    else:
        form = CreateUserForm()
    error_messages = form.errors.as_data()
    context = {'registerform': form, 'errors': error_messages}
    return render(request, 'register.html', context=context)

def my_login(request):
```

```

form = LoginForm()
if request.method == 'POST':
    form = LoginForm(request, data=request.POST)
    if form.is_valid():
        username = request.POST.get('username')
        password = request.POST.get('password')
        user = authenticate(request, username=username, password=password)
        if user is not None:
            auth.login(request, user)
            return redirect("/profile")

context = {'loginform':form}
return render(request, 'my-login.html', context=context)

def user_logout(request):
    auth.logout(request)
    return redirect("/")

@csrf_exempt
def add_user_movie(request):
    if request.method == 'POST':
        movie_id = request.POST.get('movie_id')
        rating = request.POST.get('rating')
        if movie_id and rating:
            UserMovie.objects.create(movie_id=movie_id, user=request.user,
rating=rating)
            return JsonResponse({'success': True})
        else:
            return JsonResponse({'error': 'Movie ID not provided'}, status=400)
    else:
        return JsonResponse({'error': 'Method not allowed'}, status=405)

```

urls.py

```

urlpatterns = [
    path('', index),
    path('profile', profile, name="profile"),
    path('watched', watched),
    path('register', register, name="register"),
    path('my-login', my_login, name="my-login"),
    path('user-logout', user_logout, name="user-logout"),
    path('add_user_movie/', add_user_movie, name='add_user_movie'),
    path('rec', rec, name='rec')
]

```

models.py

```

class UserMovie(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    movie_id = models.IntegerField()
    rating = models.FloatField()

def get_user_movies(user_id):
    user_movies = UserMovie.objects.filter(user_id=user_id)
    return user_movies

```

forms.py

```
class CreateUserForm(UserCreationForm):
    first_name = forms.CharField(max_length=30, required=True)
    last_name = forms.CharField(max_length=30, required=True)
    email = forms.EmailField(max_length=254, required=True)

    class Meta:
        model = User
        fields = ['first_name', 'last_name', 'username', 'email', 'password1',
'password2']

class LoginForm(AuthenticationForm):
    username = forms.CharField(widget=TextInput())
    password = forms.CharField(widget=PasswordInput())
```

recommendations.py

```
import os
import pandas as pd
from django.conf import settings
from sklearn.metrics.pairwise import cosine_similarity
from .models import UserMovie

def get_combined_ratings(user_id):
    # Шлях до файлу з рейтингами
    ratings_file_path = os.path.join(settings.BASE_DIR, 'data', 'ratings.csv')

    # Отримання рейтингів користувача з бази даних
    user_movies = UserMovie.objects.filter(user_id=user_id).values('user_id',
'movie_id', 'rating')
    user_movies_df = pd.DataFrame(list(user_movies))

    # Завантаження рейтингів з файлу
    ratings_df = pd.read_csv(ratings_file_path)

    # Об'єднання рейтингів користувача та загальних рейтингів
    combined_df = pd.concat([user_movies_df, ratings_df], ignore_index=True)

    # Обчислення середніх рейтингів для кожного користувача та фільму
    combined_df = combined_df.groupby(['user_id', 'movie_id'],
as_index=False).rating.mean()

    return combined_df

def build_user_movie_matrix(combined_df):
    # Побудова матриці "користувач-фільм"
    user_movie_matrix = combined_df.pivot(index='user_id', columns='movie_id',
values='rating')
    user_movie_matrix = user_movie_matrix.fillna(0)
    return user_movie_matrix

def calculate_user_similarity(user_movie_matrix):
```

```

# Обчислення подібності між користувачами за допомогою косинусної схожості
user_similarity = cosine_similarity(user_movie_matrix)
return user_similarity

def get_recommendations(user_id, user_similarity, user_movie_matrix,
n_recommendations=5):
    # Отримання індексу користувача в матриці "користувач-фільм"
    user_index = user_movie_matrix.index.get_loc(user_id)
    similar_users = list(enumerate(user_similarity[user_index]))
    # Сортуювання подібних користувачів за спаданням подібності
    similar_users = sorted(similar_users, key=lambda x: x[1], reverse=True)[1:]

    recommended_movies = {}
    user_mean_rating = user_movie_matrix.iloc[user_index].mean()

    # Рекомендація фільмів на основі подібних користувачів
    for similar_user in similar_users: # Цикл проходиться по кожному подібному
користувачу відсортованому за спаданням подібності.
        similar_user_index = similar_user[0]
        similarity_score = similar_user[1]

        user_ratings = user_movie_matrix.iloc[similar_user_index] # Отримання
рейтингів фільмів, які оцінив поточний подібний користувач.
        user_mean_rating_similar = user_ratings.mean()

        for movie_id, rating in user_ratings.items(): # Цикл проходиться по кожному
фільму, який оцінив поточний подібний користувач.
            if user_movie_matrix.iloc[user_index,
user_movie_matrix.columns.get_loc(movie_id)] == 0: # Перевірка, чи фільм ще не був
оцінений користувачем.
                if movie_id not in recommended_movies:
                    # Додавання фільму до списку рекомендованих з оцінкою,
обчисленою за допомогою формули колаборативної фільтрації із підходом user-based.
                    recommended_movies[movie_id] = user_mean_rating +
(similarity_score * (rating - user_mean_rating_similar)) / abs(similarity_score)
                else:
                    recommended_movies[movie_id] += user_mean_rating +
(similarity_score * (rating - user_mean_rating_similar)) / abs(similarity_score)

            # Сортуювання рекомендованих фільмів за оцінкою
            recommended_movies = sorted(recommended_movies.items(), key=lambda x: x[1],
reverse=True)
        recommended_movie_ids = [movie_id for movie_id, _ in
recommended_movies][:n_recommendations]

    return recommended_movie_ids

def recommend_movies_for_user(user_id, n_recommendations=5):
    # Отримання комбінованих рейтингів для користувача
    combined_df = get_combined_ratings(user_id)
    # Побудова матриці "користувач-фільм"
    user_movie_matrix = build_user_movie_matrix(combined_df)
    # Обчислення подібності між користувачами
    user_similarity = calculate_user_similarity(user_movie_matrix)
    # Отримання рекомендацій для користувача

```

```

    recommendations = get_recommendations(user_id, user_similarity,
user_movie_matrix, n_recommendations)
    return recommendations

```

index.js

```

const movieContainer = document.getElementById('movie-container');
const nextButton = document.getElementById('next-button');
const watchedButton = document.getElementById('watched-button');
const ratingInput = document.getElementById('ratingInput');
const finale = document.getElementById('finale');
const finale2 = document.getElementById('finale2');
const btt = document.getElementById('btt');
const apiKey = '7f540cdc422c17adc3781b8227d0b71c';

let currentIndex = 0;
let popularMovies = [];
let mode = '';

function startQuickRecommendations() {
    mode = 'popular';
    movieContainer.style.display = "flex";
    document.getElementById("container").style.display = "none";
    btt.style.display = "flex";
    watchedButton.style.display = "none";
    ratingInput.style.display = "none";
    initialize();
}

function startPersonalRecommendations() {
    mode = 'top_rated';
    movieContainer.style.display = "flex";
    document.getElementById("container").style.display = "none";
    btt.style.display = "flex";
    initialize();
}

async function fetchMovieDetails(movieId) {
    const response = await
fetch(`https://api.themoviedb.org/3/movie/${movieId}?api_key=${apiKey}&language=en-US`);
    const data = await response.json();
    return data;
}

async function fetchPopularMovies() {
    const response = await
fetch(`https://api.themoviedb.org/3/movie/${mode}?api_key=${apiKey}&page=1`);
    const data = await response.json();
    return data.results.map(movie => movie.id);
}

async function showNextMovie() {
    watchedButton.disabled = false;
    if (currentIndex >= popularMovies.length) {

```



```

    if (mode === 'popular') {
      finale.style.display = 'flex';
      movieContainer.style.display = 'none';
      btt.style.display = 'none';
      return;
    } else {
      finale2.style.display = 'flex';
      movieContainer.style.display = 'none';
      btt.style.display = 'none';
      return;
    }
  }
  const movieId = popularMovies[currentIndex];
  const movieDetails = await fetchMovieDetails(movieId);
  const genres = movieDetails.genres.map(genre => genre.name).join(', ');
  const country = movieDetails.production_countries.map(c =>
c.name).join(', ');
  const roundedRating = movieDetails.vote_average.toFixed(1);
  movieContainer.innerHTML = `
    
    <div class="info">
      <h1>${movieDetails.title}</h1>
      <h4>"${movieDetails.tagline}"</h4>
      <h3>${movieDetails.release_date} | ${movieDetails.runtime} min.
</h3>

      <h3>${country}</h3>
      <h3>Genres: ${genres}</h3>
      <h3>Rating: ${roundedRating}</h3>
      <p>Description: ${movieDetails.overview}</p>
    </div>
  `;
  currentIndex++;
}
nextButton.addEventListener('click', showNextMovie);
async function initialize() {
  popularMovies = await fetchPopularMovies();
  showNextMovie();
}
document.getElementById('watched-form').addEventListener('submit',
function(event) {
  event.preventDefault();
  ratingInput.style.display = 'block';

  const movieId = popularMovies[currentIndex - 1];
  document.getElementById('movieIdInput').value = movieId;
  const formData = new FormData(this);
  const rating = formData.get('rating');

  if (!rating) {
    alert('Please select a rating before submitting.');
```

```

        body: formData
    })
    .then(response => {
        if (!response.ok) {
            throw new Error('An error occurred during the request.');
```

```

        }
        alert('Successfully added to the watched list!');
        document.querySelector('input[name="rating"]:checked').checked = false;
        watchedButton.disabled = true;
    })
    .catch(error => {
        console.error('Error:', error);
    });
});
```

watched.js

```

const movieContainer = document.getElementById('movieContainer');
const apiKey = '7f540cdc422c17adc3781b8227d0b71c';
async function fetchMovieDetails(movieIds) {
    const movieDetailsPromises = movieIds.map(async movieId => {
        const response = await
fetch(`https://api.themoviedb.org/3/movie/${movieId}?api_key=${apiKey}&language=en-US`);
        return response.json();
    });

    return Promise.all(movieDetailsPromises);
}

async function showAllMovies() {
    const userMovies = JSON.parse("{{ user_movies_json|escapejs }}");
    console.log(userMovies)
    const movieIds = userMovies.map(movie => movie.fields.movie_id);
    console.log(movieIds)
    const movieDetails = await fetchMovieDetails(movieIds);
    movieContainer.innerHTML = movieDetails.map(movie => {
        return `
            <div class="movie">
                
                <h1>${movie.title}</h1>
            </div>
        `;
    }).join('');
}
showAllMovies();
```